

UVW-KEM

Algorithm Description Document

Submitters:

- Fangguo Zhang (Sun Yat-sen University, School of Computer Science and Engineering)
- Yang Yang (Jiangxi Normal University, School of Artificial Intelligence (School of Digital Industry))
- Kuojie Xiao (Sun Yat-sen University, School of Computer Science and Engineering)
- Xiaofeng Chen (Xidian University, School of Cryptology)
- Chen Li (Sun Yat-sen University, School of Computer Science and Engineering)
- Haibo Tian (Sun Yat-sen University, School of Computer Science and Engineering)

April 29, 2026

Contents

1	Algorithm Description	6
1.1	Preliminaries	6
1.1.1	Notation	6
1.1.2	Coding Theory	6
1.1.3	ReedSolomon Codes and List Decoding Algorithms	9
1.1.4	Public Key Encryption	10
1.1.5	Key Encapsulation Mechanism	11
1.2	Algorithm Procedure	11
1.2.1	UVW Public Key Encryption Scheme	11
1.2.2	UVW Key Encapsulation Mechanism	14
1.3	Algorithm Instances and Parameters	15
2	Design Rationale	16
2.1	Underlying Hard Problems	16
2.2	Decryption Failure Probability Analysis	16
2.3	Parameter Selection Rationale	18
3	Security Analysis	18
3.1	Security Proof	18
3.2	Concrete Security	22
3.2.1	Classical Security Analysis	22
3.2.2	Quantum Security Analysis	29
4	Performance Analysis	30
4.1	Theoretical Performance Analysis	30
4.2	Performance Evaluation	31
5	Features	33
	References	33

List of Figures

1	Game 0: The real IND-CPA game for encrypting message	19
2	Game 1: replaces the public key matrix $\mathbf{G}_{\text{pk}}$ with a random matrix $\mathbf{G}$	20
3	Game 2: replaces $\text{Hash}(\mathbf{r}, \mathbf{e})$ with a random bit string $\mathbf{s}$	20
4	Game 3: The real IND-CPA game for encrypting message $\mathbf{m}_1$	21
5	Normalized logarithm of the density of $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords as a function of relative weight w/n	25
6	Comparison of expected numbers of $(\mathbf{u}, \mathbf{u})\mathbf{D}$ and non- $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords as a function of weight t for UVW128.	28
7	Comparison for weights $t \in [330, 430]$ for UVW128.	28

List of Tables

1	Recommended Parameter Sets for the UVW KEM	16
2	Transmission and storage overhead of UVW KEM (in Bytes).	31
3	Performance results of the UVW KEM reference implementation.	32
4	Performance results of the UVW KEM optimized implementation.	32

List of Algorithms

1	ListDec.RS($\mathbf{G}, \mathbf{c}, w$): Reed-Solomon List Decoding Algorithm Capable of Correcting w Errors	10
2	UVW.PKE.Setup(1^λ): Initialization Algorithm	12
3	UVW.PKE.KeyGen($params$): Key Generation Algorithm	12
4	UVW.PKE.Enc($pk, \mathbf{m}$): Encryption Algorithm	13
5	UVW.PKE.Dec($sk, \mathbf{c}$): Decryption Algorithm	13
6	UVW.KEM.Setup(1^λ): Initialization Algorithm	15
7	UVW.KEM.KeyGen($params$): Key Generation Algorithm	15
8	UVW.KEM.Encap(pk): Encapsulation Algorithm	15
9	UVW.KEM.Decap($sk, (\mathbf{c}_1, \mathbf{c}_2, \mathbf{d})$): Decapsulation Algorithm	15

1 Algorithm Description

1.1 Preliminaries

1.1.1 Notation

$\mathbb{F}_q$ denotes the finite field of size q . Bold lowercase (e.g., $\mathbf{x}$) and uppercase (e.g., $\mathbf{H}$) letters represent row vectors and matrices, respectively. $\mathbf{x}^\top$ ($\mathbf{H}^\top$) denotes the transpose of vector $\mathbf{x}$ (matrix $\mathbf{H}$). Let $m, n > m$ be positive integers, $[n]$ and $[m, n]$ denote the integer sets $\{1, 2, \dots, n\}$ and $\{m, m+1, \dots, n\}$, S_n denotes the set of permutations of order n , and $\mathcal{DP}(n)$ denotes the set of all monomial matrices (products of permutation matrices and invertible diagonal matrices) of order n over $\mathbb{F}_q$. For $\mathbf{x} \in \mathbb{F}_q^n$, $\mathbf{x}(i)$ denotes the i -th component of $\mathbf{x}$, sometimes also written as x_i , i.e., $\mathbf{x} = (x_i)_{i \in [n]}$. $|\mathbf{x}|$ denotes the Hamming weight of $\mathbf{x}$, i.e., $|\mathbf{x}| = |\{i \in [n] \mid \mathbf{x}(i) \neq 0\}|$, sometimes also written as $wt(\mathbf{x})$. $\lfloor a \rfloor$ denotes the floor of the rational number a . $negl(\lambda)$ denotes a negligible function in λ . $\mathcal{E}_{n,w}$ denotes the set of all vectors in $\mathbb{F}_q^n$ with Hamming weight w , whose cardinality is $\binom{n}{w}(q-1)^w$. $s \xleftarrow{\$} S$ denotes that element s is selected uniformly at random from set S . Let X be a random variable. $\mathbb{P}(X = A)$ denotes the probability of event A , and $\mathbb{E}(X)$ is its expectation. PRNG denotes a pseudo-random number generator.

1.1.2 Coding Theory

An $[n, k]$ linear code $\mathcal{C}$ over the finite field $\mathbb{F}_q$ can be viewed as a k -dimensional subspace of the n -dimensional vector space $\mathbb{F}_q^n$. Vectors in $\mathcal{C}$ are called codewords, where n is the length of the code and k is its dimension. Similar to how a basis represents a linear space, a linear code $\mathcal{C}$ can be represented by a basis composed of its codewords; the matrix formed by this basis is called a generator matrix of $\mathcal{C}$.

Definition 1 (Generator Matrix and Parity-Check Matrix). *Let $\mathcal{C}$ be an $[n, k]$ linear code over $\mathbb{F}_q$. A generator matrix of $\mathcal{C}$ is a full-rank $k \times n$ matrix over $\mathbb{F}_q$ whose row vectors form a basis of $\mathcal{C}$. If*

$$\mathcal{C} = \{\mathbf{x}\mathbf{G} \mid \mathbf{x} \in \mathbb{F}_q^k\}, \quad (1)$$

then $\mathbf{G}$ is a generator matrix of $\mathcal{C}$. A parity-check matrix of $\mathcal{C}$ is a full-rank $(n-k) \times n$ matrix over $\mathbb{F}_q$. If

$$\forall \mathbf{x} \in \mathcal{C} \Leftrightarrow \mathbf{x}\mathbf{H}^\top = \mathbf{0}, \quad (2)$$

then $\mathbf{H}$ is a parity-check matrix of $\mathcal{C}$.

Generator and parity-check matrices of a code $\mathcal{C}$ are not unique. For any generator matrix $\mathbf{G}$ and parity-check matrix $\mathbf{H}$ of $\mathcal{C}$, $\mathbf{G}\mathbf{H}^\top = \mathbf{0}$. An information set of an $[n, k]$ linear code $\mathcal{C}$ is a set of k coordinate indices, denoted $I \subset [n]$, corresponding to k linearly independent columns of a generator matrix $\mathbf{G}$. Its complement corresponds to r ($r = n - k$) linearly independent columns of a parity-check matrix $\mathbf{H}$. Given a parity-check matrix $\mathbf{H} \in \mathbb{F}_q^{r \times n}$ and an information set I , for any vectors $\mathbf{s} \in \mathbb{F}_q^r$ and $\mathbf{x} \in \mathbb{F}_2^n$, there exists a unique vector $\mathbf{e} \in \mathbb{F}_q^n$ such that $\mathbf{e}\mathbf{H}^\top = \mathbf{s}$ and $\mathbf{x}_I = \mathbf{e}_I$.

Definition 2 ($(U, U+V)$ Code). *An $(U, U+V)$ code over $\mathbb{F}_q$ is constructed from two linear codes U and V . Let n, k_1 and k_2 be three positive integers, where n is even. Let U be an $[\frac{n}{2}, k_1]$ linear code and V be an $[\frac{n}{2}, k_2]$ linear code. Then the generator matrix of the $(U, U+V)$ code is defined as*

$$\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{0} & \mathbf{G}_V \end{pmatrix}, \quad (3)$$

where $\mathbf{G}_U$ and $\mathbf{G}_V$ are generator matrices of U and V , respectively.

Building upon the $(U, U+V)$ code, we propose an $(U+V, U+W)$ code, formally defined below.

Definition 3 ($(U+V, U+W)$ Code). An $(U+V, U+W)$ code over $\mathbb{F}_q$ is constructed from three linear codes U, V and W . Let n, k_1 and k_2 be three positive integers, where n is even. Let U be an $[\frac{n}{2}, k_1]$ linear code, and V and W be $[\frac{n}{2}, k_2]$ linear codes. Let $\mathbf{G}_U, \mathbf{G}_V$ and $\mathbf{G}_W$ be generator matrices of U, V and W , respectively, with $\mathbf{G}_V - \mathbf{G}_W$ being a full-rank matrix. Define the generator matrix as

$$\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix}, \quad (4)$$

Then the $[n, k]$ linear code ($k = k_1 + k_2$) generated by $\mathbf{G}$ is called an $(U+V, U+W)$ code.

Next, we define the generalized $(U+V, U+W)$ code, which is the code used in our encryption scheme.

Definition 4 (Generalized $(U+V, U+W)$ Code). A generalized $(U+V, U+W)$ code over $\mathbb{F}_q$ is constructed from three linear codes U, V and W . Let n, k_1 and k_2 be three positive integers, where n is even. Let U be an $[\frac{n}{2}, k_1]$ linear code, and V and W be $[\frac{n}{2}, k_2]$ linear codes. Let $\mathbf{D} \in \mathbb{F}_q^{n \times n}$ be a random monomial matrix, and $\mathbf{G}_U, \mathbf{G}_V$ and $\mathbf{G}_W$ be generator matrices of U, V and W , respectively, with $\mathbf{G}_V - \mathbf{G}_W$ being a full-rank matrix. Define the generator matrix as

$$\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D}, \quad (5)$$

Then the $[n, k]$ linear code ($k = k_1 + k_2$) generated by $\mathbf{G}$ is called a generalized $(U+V, U+W)$ code.

Corollary 1. Let $\mathcal{C}$ be a generalized $[n, k]$ $(U+V, U+W)$ code with generator matrix $\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D}$.

Define its dual code $\mathcal{C}^\perp$ as

$$\mathcal{C}^\perp = \{\mathbf{y} \in \mathbb{F}_q^n \mid \forall \mathbf{x} \in \mathcal{C}, \langle \mathbf{x}, \mathbf{y} \rangle = 0\}, \quad (6)$$

Then $\mathcal{C}^\perp$ is a generalized $[n, n-k]$ $(U+V, U+W)$ code.

Proof. The proof of Corollary 1 is as follows. From Definition 4, $\mathbf{G}_U, \mathbf{G}_V$ and $\mathbf{G}_W$ are generator matrices of U, V and W , respectively, and $\mathbf{G}_\Delta = \mathbf{G}_W - \mathbf{G}_V$ has full row rank. To construct the parity-check matrix $\mathbf{H}$ of $\mathcal{C}$ (i.e., the generator matrix of the dual code $\mathcal{C}^\perp$), we define the following three matrices based directly on the dual relationships of the underlying sub-codes: 1) Let $\mathbf{H}_U \in \mathbb{F}_q^{(n/2-k_1) \times n/2}$ be the parity-check matrix of sub-code U , satisfying $\mathbf{G}_U \mathbf{H}_U^T = \mathbf{0}$; 2) Let $\mathbf{H}_\Delta \in \mathbb{F}_q^{(n/2-k_2) \times n/2}$ be the parity-check matrix of the sub-code generated by $\mathbf{G}_\Delta$, satisfying $(\mathbf{G}_W - \mathbf{G}_V) \mathbf{H}_\Delta^T = \mathbf{0}$. 3) Since $\mathbf{G}_\Delta$ has full row rank, the non-homogeneous linear system $\mathbf{G}_\Delta \mathbf{Y}^T = -\mathbf{G}_V \mathbf{H}_U^T$ admits infinitely many solutions. Exploiting the degrees of freedom in the solution space, we can select a particular solution matrix denoted $\mathbf{H}_Y \in \mathbb{F}_q^{(n/2-k_1) \times n/2}$ such that $\mathbf{H}_Y$ itself has full rank and the derived matrix difference $\mathbf{H}_U - \mathbf{H}_Y$ also has full rank.

Based on the above components, we construct a candidate parity-check matrix $\mathbf{H}$ as follows:

$$\mathbf{H} = \begin{pmatrix} \mathbf{H}_U - \mathbf{H}_Y & \mathbf{H}_Y \\ -\mathbf{H}_\Delta & \mathbf{H}_\Delta \end{pmatrix} (\mathbf{D}^{-1})^T,$$

We need to verify $\mathbf{GH}^T = \mathbf{0}$. Expanding the matrix multiplication:

$$\begin{aligned} \mathbf{GH}^T &= \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D} \cdot \mathbf{D}^{-1} \begin{pmatrix} (\mathbf{H}_U - \mathbf{H}_Y)^T & -\mathbf{H}_\Delta^T \\ \mathbf{H}_Y^T & \mathbf{H}_\Delta^T \end{pmatrix} \\ &= \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \begin{pmatrix} \mathbf{H}_U^T - \mathbf{H}_Y^T & -\mathbf{H}_\Delta^T \\ \mathbf{H}_Y^T & \mathbf{H}_\Delta^T \end{pmatrix}, \end{aligned}$$

Computing the four blocks of the resulting matrix separately:

1. Top-left block: $\mathbf{G}_U(\mathbf{H}_U^T - \mathbf{H}_Y^T) + \mathbf{G}_U\mathbf{H}_Y^T = \mathbf{G}_U\mathbf{H}_U^T - \mathbf{G}_U\mathbf{H}_Y^T + \mathbf{G}_U\mathbf{H}_Y^T = \mathbf{G}_U\mathbf{H}_U^T = \mathbf{0}$;
2. Top-right block: $-\mathbf{G}_U\mathbf{H}_\Delta^T + \mathbf{G}_U\mathbf{H}_\Delta^T = \mathbf{0}$;
3. Bottom-right block: $-\mathbf{G}_V\mathbf{H}_\Delta^T + \mathbf{G}_W\mathbf{H}_\Delta^T = \mathbf{G}_\Delta\mathbf{H}_\Delta^T = \mathbf{0}$;
4. Bottom-left block: $\mathbf{G}_V(\mathbf{H}_U^T - \mathbf{H}_Y^T) + \mathbf{G}_W\mathbf{H}_Y^T = \mathbf{G}_V\mathbf{H}_U^T + (\mathbf{G}_W - \mathbf{G}_V)\mathbf{H}_Y^T$. Substituting the defining equation for $\mathbf{H}_Y$, $(\mathbf{G}_W - \mathbf{G}_V)\mathbf{H}_Y^T = -\mathbf{G}_V\mathbf{H}_U^T$, yields $\mathbf{G}_V\mathbf{H}_U^T - \mathbf{G}_V\mathbf{H}_U^T = \mathbf{0}$. Thus, $\mathbf{GH}^T = \mathbf{0}$ is rigorously proved.

Next, we verify the rank of the parity-check matrix $\mathbf{H}$. Consider the core block matrix $\mathbf{M} = \begin{pmatrix} \mathbf{H}_U - \mathbf{H}_Y & \mathbf{H}_Y \\ -\mathbf{H}_\Delta & \mathbf{H}_\Delta \end{pmatrix}$. Adding the right column block to the left column block (elementary column operations do not change the rank of a matrix) yields:

$$\mathbf{M} \xrightarrow{C_1 \leftarrow C_1 + C_2} \begin{pmatrix} (\mathbf{H}_U - \mathbf{H}_Y) + \mathbf{H}_Y & \mathbf{H}_Y \\ -\mathbf{H}_\Delta + \mathbf{H}_\Delta & \mathbf{H}_\Delta \end{pmatrix} = \begin{pmatrix} \mathbf{H}_U & \mathbf{H}_Y \\ \mathbf{0} & \mathbf{H}_\Delta \end{pmatrix},$$

This is transformed into a block upper triangular matrix, whose rank is the sum of the ranks of its diagonal blocks:

$$\text{Rank}(\mathbf{M}) = \text{Rank}(\mathbf{H}_U) + \text{Rank}(\mathbf{H}_\Delta) = (n/2 - k_1) + (n/2 - k_2) = n - (k_1 + k_2) = n - k,$$

Since $(\mathbf{D}^{-1})^T$ is an invertible matrix, right multiplication does not change the rank, so $\mathbf{H} = \mathbf{M}(\mathbf{D}^{-1})^T$ has full rank $n - k$.

By permuting the block rows of matrix $\mathbf{H}$ and absorbing the negative signs (via diagonal permutation), it can be transformed into the standard form of a generator matrix for a generalized $(U + V, U + W)$ code. Therefore, the dual code $\mathcal{C}^\perp$ is also a generalized $(U + V, U + W)$ code. $\square$ $\square$

We define $\mathbf{H}_X = \mathbf{H}_U - \mathbf{H}_Y$ and $\mathbf{H}_Z = \mathbf{H}_\Delta$, and observe that $(\mathbf{D}^{-1})^T$ is still a monomial matrix. Then the parity-check matrix can be equivalently rewritten as:

$$\mathbf{H} = \begin{pmatrix} \mathbf{H}_X & \mathbf{H}_Y \\ -\mathbf{H}_Z & \mathbf{H}_Z \end{pmatrix} \mathbf{D} \tag{7}$$

From Definition 1, any linear code can be represented by either a generator matrix $\mathbf{G}$ or a parity-check matrix $\mathbf{H}$. For convenience, unless otherwise specified, we will use the generator matrix defined as

$$\mathbf{G} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D}, \quad (8)$$

to represent a generalized $[n, k]$ ($U + V, U + W$) code, where $\mathbf{G}_U \in \mathbb{F}_q^{k_1 \times n/2}$, $\mathbf{G}_V, \mathbf{G}_W \in \mathbb{F}_q^{k_2 \times n/2}$, and $k = k_1 + k_2$.

1.1.3 ReedSolomon Codes and List Decoding Algorithms

Definition 5 (Reed-Solomon Code). *Let $\vec{\alpha} = (\alpha_1, \alpha_2, \dots, \alpha_n)$, where α_i ($i \in [n]$) are distinct non-zero elements in $\mathbb{F}_q$. Then the k -dimensional Reed-Solomon code $\mathcal{C}$ defined by $\vec{\alpha}$ is*

$$\mathcal{C} = \{(f(\alpha_1), f(\alpha_2), \dots, f(\alpha_n)) \mid \deg(f) < k\}. \quad (9)$$

Let $f_j(x) = x^j$ ($j = 0, \dots, k-1$). Then the matrix

$$\mathbf{G} = \begin{pmatrix} f_0(\alpha_1) & f_0(\alpha_2) & \cdots & f_0(\alpha_n) \\ f_1(\alpha_1) & f_1(\alpha_2) & \cdots & f_1(\alpha_n) \\ \vdots & \vdots & \ddots & \vdots \\ f_{k-1}(\alpha_1) & f_{k-1}(\alpha_2) & \cdots & f_{k-1}(\alpha_n) \end{pmatrix} = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ \alpha_1 & \alpha_2 & \cdots & \alpha_n \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_1^{k-1} & \alpha_2^{k-1} & \cdots & \alpha_n^{k-1} \end{pmatrix} \quad (10)$$

is a generator matrix of $\mathcal{C}$. The dual code $\mathcal{C}^\perp$ is an $[n, n-k]$ Reed-Solomon code. A Reed-Solomon code can also be defined by its generator polynomial $g(x)$. Let $\alpha \in \mathbb{F}_q$ be a primitive field element. If the code is required to have minimum distance d , its generator polynomial can be expressed as $g(x) = \prod_{i=1}^{d-1} (x - \alpha^i)$, and its parity-check matrix is defined as

$$\mathbf{H} = \begin{pmatrix} \alpha^{n-1} & \alpha^{n-2} & \cdots & \alpha^2 & \alpha & 1 \\ (\alpha^2)^{n-1} & (\alpha^2)^{n-2} & \cdots & (\alpha^2)^2 & (\alpha^2) & 1 \\ \vdots & \vdots & & \vdots & \vdots & \vdots \\ (\alpha^{d-1})^{n-1} & (\alpha^{d-1})^{n-2} & \cdots & (\alpha^{d-1})^2 & (\alpha^{d-1}) & 1 \end{pmatrix}. \quad (11)$$

A Reed-Solomon code is a Maximum Distance Separable (MDS) code; its minimum distance, which is also the weight of its minimum weight codeword, is $d = n - k + 1$. List decoding algorithms are able to correct more errors than half of the minimum distance. Below we present the Guruswami–Sudan list decoding algorithm for Reed–Solomon codes [1]. Regarding the selection of decoding parameters in the Guruswami–Sudan algorithm, this paper follows the improved parameter selection strategy proposed by Wang [2] in order to achieve better decoding performance.

Algorithm 1 ListDec.RS($\mathbf{G}, \mathbf{c}, w$): Reed-Solomon List Decoding Algorithm Capable of Correcting w Errors

Input: RS code generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$, received vector $\mathbf{c} \in \mathbb{F}_q^n$, decoding radius w

- 1: Extract the support set $\mathbf{x} = (x_1, \dots, x_n)$ from the structure of $\mathbf{G}$.
- 2: Compute Guruswami-Sudan decoding parameters: interpolation multiplicity M and maximum y -degree L based on w, n, k .
- 3: **Interpolation Step:** Find a non-zero bivariate polynomial $Q(x, y) \in \mathbb{F}_q[x, y]$ with minimal $(1, k-1)$ -weighted degree, such that for all $i \in \{1, \dots, n\}$, the point (x_i, c_i) has multiplicity at least M .
- 4: **Factorization Step:** Compute all univariate polynomial roots of $Q(x, y)$ satisfying the degree constraints to obtain a set of candidate message polynomials:

$$\mathcal{F} = \{f(x) \in \mathbb{F}_q[x] \mid Q(x, f(x)) = 0, \deg(f) < k\}$$

- 5: Initialize valid output set $\mathcal{V} = \emptyset$.
 - 6: **Selection Step:** For each $f(x) \in \mathcal{F}$:
 - 7: Extract coefficients of $f(x)$ to form the candidate message vector $\mathbf{r} = (f_0, f_1, \dots, f_{k-1}) \in \mathbb{F}_q^k$.
 - 8: Compute the corresponding error vector $\mathbf{e} = \mathbf{c} - \mathbf{r}\mathbf{G}$.
 - 9: If its Hamming weight $\text{wt}(\mathbf{e}) \leq w$, set $\mathcal{V} = \mathcal{V} \cup \{(\mathbf{r}, \mathbf{e})\}$.
 - 10: **return** $\mathcal{V}$
- Output:** Set of valid "message-error" pairs $\mathcal{V} = \{(\mathbf{r}, \mathbf{e})\}$
-

1.1.4 Public Key Encryption

Definition 6 (Public Key Encryption). *A typical public-key encryption scheme consists of the following three polynomial-time algorithms: KeyGen, Encrypt, and Decrypt.*

1. **KeyGen**(1^λ): *A probabilistic polynomial-time algorithm that, given the security parameter 1^λ , outputs a public/private key pair $(\mathbf{pk}, \mathbf{sk})$, denoted $(\mathbf{pk}, \mathbf{sk}) \leftarrow \text{KeyGen}(1^\lambda)$. The public key $\mathbf{pk}$ is publicly available for encrypting messages, while the private key $\mathbf{sk}$ is known only to the holder for decrypting messages.*
2. **Encrypt**($\mathbf{pk}, \mathbf{m}$): *A probabilistic polynomial-time algorithm that, given a message $\mathbf{m}$ and public key $\mathbf{pk}$, outputs a ciphertext $\mathbf{c}$, denoted $\mathbf{c} \leftarrow \text{Encrypt}(\mathbf{pk}, \mathbf{m})$.*
3. **Decrypt**($\mathbf{sk}, \mathbf{c}$): *A deterministic polynomial-time algorithm that, given a ciphertext $\mathbf{c}$ and private key $\mathbf{sk}$, outputs the corresponding plaintext message $\mathbf{m}$. If the ciphertext $\mathbf{c}$ is invalid, it outputs the termination symbol $\perp$, denoted $\mathbf{m} = \text{Decrypt}(\mathbf{sk}, \mathbf{c})$.*

Correctness: For any key pair $(\mathbf{pk}, \mathbf{sk}) \leftarrow \text{KeyGen}(1^\lambda)$ and any message $\mathbf{m}$, the following equation always holds:

$$\text{Decrypt}(\mathbf{sk}, \text{Encrypt}(\mathbf{pk}, \mathbf{m})) = \mathbf{m}. \quad (12)$$

This means that, given the public and private keys, a message encrypted using the encryption algorithm can be accurately decrypted back to the original message via the decryption algorithm. In other words, for any message $\mathbf{m}$, the ciphertext $\mathbf{c}$ generated after encryption must be decrypted back to the original message $\mathbf{m}$.

1.1.5 Key Encapsulation Mechanism

Definition 7 (Key Encapsulation Mechanism). *A key encapsulation mechanism consists of the following three polynomial-time algorithms: KeyGen, Encaps, and Decaps.*

1. **KeyGen**(1^λ): *A probabilistic polynomial-time algorithm that, given the security parameter 1^λ , outputs a public/private key pair (pk, sk) , denoted $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$. The public key pk is publicly available for encrypting messages, while the private key sk is known only to the holder for decrypting messages.*
2. **Encaps**($\text{pk}, 1^\lambda$): *A probabilistic polynomial-time algorithm that, given the public key pk and security parameter 1^λ , outputs a ciphertext $\mathbf{c}$ and a session key K , denoted $(\mathbf{c}, K) \leftarrow \text{Encaps}(\text{pk}, 1^\lambda)$.*
3. **Decaps**($\text{sk}, \mathbf{c}$): *A deterministic polynomial-time algorithm that, given the private key sk and a ciphertext $\mathbf{c}$, outputs a session key K or the termination symbol $\perp$ indicating decryption failure, denoted $K = \text{Decaps}(\text{sk}, \mathbf{c})$.*

1.2 Algorithm Procedure

In this section, we detail the specific construction of the UVW key encapsulation mechanism. For ease of understanding, we first present the UVW public-key encryption scheme, which is secure against chosen-plaintext attacks.

1.2.1 UVW Public Key Encryption Scheme

We present the concrete construction of the UVW public key encryption scheme. The UVW PKE scheme is built upon the generalized $(U + V, U + W)$ code and comprises four algorithms: **Setup**, **KeyGen**, **Enc**, and **Dec**.

In the key generation algorithm, we introduce counters i_1, i_2, i_3 and store them together with the seed *seed* as the secret key. The main purpose is to enable the decryptor to deterministically reconstruct exactly the same matrices as those generated during encryption in a one-shot manner, thereby eliminating the verification steps in the key generation phase that rely on randomized trial-and-error, in particular the full-rank checks of matrices, and thus accelerating decryption. More specifically, each matrix is generated by concatenating the seed with a counter and a fixed label, and then expanding it via a PRNG to produce random bytes. Every 2 bytes are parsed into a 16-bit integer, and its lower 9 bits are taken as a candidate value (under the UVW128 parameter set). If the candidate value is smaller than the field size q , it is accepted as an element of $\mathbb{F}_q$; otherwise, the procedure is repeated. For $\mathbf{G}_U$ and $\mathbf{G}_W$, the resulting elements are filled in row-major order into a $k_1 \times n/2$ or $k_2 \times n/2$ matrix, respectively. Since the decryptor directly uses the stored counter values, it does not need to perform the full-rank check and can directly reconstruct the same full-rank matrices as the encryptor. For the monomial matrix $\mathbf{D}$, a random permutation of length n and an array of nonzero random coefficients of length n are generated to stored it.

For a random matrix over the finite field $\mathbb{F}_q$, the probability that an $r \times c$ matrix ($r < c$) is full rank is given by $\prod_{i=0}^{r-1} (1 - q^{i-c})$ [3], and the upper bound on the probability of being rank-deficient is approximately $q^{-(c-r+1)}$. In the key generation algorithm, assuming $(q, n, k_1, k_2) = (433, 860, 215, 215)$,

the probability that the randomly generated matrices $\mathbf{G}_U$ and $\mathbf{G}_W$ are not full rank is about 433^{-216} , which implies that generating full-rank matrices $\mathbf{G}_U$ and $\mathbf{G}_W$ succeeds almost certainly on the first attempt.

On the other hand, during key generation we need to find a monomial matrix $\mathbf{D}$ such that $\mathbf{G}_{\text{sk}} = (\mathbf{A} \parallel \mathbf{B})$ can be transformed into systematic form via row operations. This is equivalent to checking whether a random $k \times k$ matrix $\mathbf{A}$ is invertible; if $\mathbf{A}$ is invertible, then a suitable $\mathbf{D}$ has been found. Over $\mathbb{F}_q$, the probability that a randomly chosen $n \times n$ matrix is singular is $1 - \prod_{j=1}^n (1 - q^{-j})$ [3], and it is upper bounded by $\frac{1}{q-1}$. Assuming $(q, n, k) = (433, 860, 430)$, the probability that $\mathbf{A}$ is singular is approximately 0.0023, which indicates that in practice a valid $\mathbf{D}$ can be found with almost a single attempt, and hence the systematic-form transformation incurs negligible overhead.

Algorithm 2 UVW.PKE.Setup(1^λ): Initialization Algorithm

Input: Security parameter λ

- 1: Select system parameters $params = (q, n, k, k_1, k_2, w, m)$ according to the security parameter, where $k = k_1 + k_2$.
 - 2: Select a secure hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{F}_2^m$.
 - 3: Define the finite field $\mathbb{F}_q$; all subsequent operations are performed over $\mathbb{F}_q$.
-

Algorithm 3 UVW.PKE.KeyGen($params$): Key Generation Algorithm

Input: System parameters $params$

- 1: Sample a random seed $seed \in \{0, 1\}^m$; generate counters $i_1, i_2, i_3 \in \{0, 1\}^8$, all initialized to 0
- 2: Set $i_1 = i_1 + 1$, generate a matrix $\mathbf{G}_U \leftarrow \text{PRNG}(seed \parallel i_1 \parallel 1) \in \mathbb{F}_q^{k_1 \times n/2}$. If $\mathbf{G}_U$ is not full rank, repeat this step
- 3: Set $i_2 = i_2 + 1$, generate a matrix $\mathbf{G}_W \leftarrow \text{PRNG}(seed \parallel i_2 \parallel 2) \in \mathbb{F}_q^{k_2 \times n/2}$. If $\mathbf{G}_W$ is not full rank, repeat this step
- 4: Let $\mathbf{G}_{RS} \in \mathbb{F}_q^{k_2 \times n/2}$ be the generator matrix of the $[n/2, k_2]$ Reed–Solomon code $\mathcal{C}$ over the finite field $\mathbb{F}_q$, which can correct w errors and is defined by $\vec{\alpha} = (1, 2, \dots, n/2)$
- 5: Compute $\mathbf{G}_V = \mathbf{G}_{RS} + \mathbf{G}_W$
- 6: Set $i_3 = i_3 + 1$, generate a monomial matrix $\mathbf{D} \leftarrow \text{PRNG}(seed \parallel i_3 \parallel 3) \in \mathcal{DP}(n)$
- 7: Compute $\mathbf{G}_{\text{sk}} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D}$
- 8: Perform elementary row operations on $\mathbf{G}_{\text{sk}}$ and attempt to transform it into systematic form $(\mathbf{I}_k \parallel \mathbf{T})$.
If this fails, return to Step 6 and regenerate the monomial matrix $\mathbf{D}$

Output: Secret key $\text{sk} = (seed, i_1, i_2, i_3)$, public key $\text{pk} = \mathbf{T}$

Algorithm 4 UVW.PKE.Enc(pk, m): Encryption Algorithm

Input: Public key pk and message to encrypt $\mathbf{m} \in \mathbb{F}_2^m$

- 1: Randomly select vector $\mathbf{r} \xleftarrow{\$} \mathbb{F}_q^k$.
- 2: Randomly select a weight- w vector $\mathbf{e} \in \mathcal{E}_{q,n}^w$.
- 3: Set $\mathbf{G}_{\text{pk}} = (\mathbf{I}_k \parallel \mathbf{T})$.
- 4: Compute $\mathbf{c}_1 = \mathbf{r}\mathbf{G}_{\text{pk}} + \mathbf{e}$, $\mathbf{c}_2 = \mathbf{m} \oplus \mathcal{H}(\mathbf{r}, \mathbf{e})$.

Output: Ciphertext $\mathbf{c} = (\mathbf{c}_1, \mathbf{c}_2)$.

Algorithm 5 UVW.PKE.Dec(sk, c): Decryption Algorithm

Input: Secret key $\text{sk} = (\text{seed}, i_1, i_2, i_3)$, ciphertext $\mathbf{c} = (\mathbf{c}_1, \mathbf{c}_2)$

- 1: Reconstruct the full-rank matrix $\mathbf{G}_U \leftarrow \text{PRNG}(\text{seed} \parallel i_1 \parallel 1) \in \mathbb{F}_q^{k_1 \times n/2}$
- 2: Reconstruct the full-rank matrix $\mathbf{G}_W \leftarrow \text{PRNG}(\text{seed} \parallel i_2 \parallel 2) \in \mathbb{F}_q^{k_2 \times n/2}$
- 3: Reconstruct the monomial matrix $\mathbf{D} \leftarrow \text{PRNG}(\text{seed} \parallel i_3 \parallel 3) \in \mathcal{DP}(n)$
- 4: Let $\mathbf{G}_{RS} \in \mathbb{F}_q^{k_2 \times n/2}$ be the generator matrix of the $[n/2, k_2]$ Reed-Solomon code $\mathcal{C}$ over the finite field $\mathbb{F}_q$, which can correct w errors and is defined by $\vec{\alpha} = (1, 2, \dots, n/2)$
- 5: Compute $\mathbf{G}_V = \mathbf{G}_{RS} + \mathbf{G}_W$
- 6: Construct $\mathbf{G}_{\text{sk}} = \begin{pmatrix} \mathbf{G}_U & \mathbf{G}_U \\ \mathbf{G}_V & \mathbf{G}_W \end{pmatrix} \mathbf{D}$, and partition it column-wise as $(\mathbf{A} \parallel \mathbf{B})$, where $\mathbf{A}$ is the left $k \times k$ square submatrix
- 7: Compute $\mathbf{c}_1 \mathbf{D}^{-1} = (\mathbf{c}_{11} \parallel \mathbf{c}_{12})$
- 8: Compute $(\mathbf{r}_2, \bar{\mathbf{e}}) = \text{ListDec.RS}(\mathbf{G}_{RS}, \mathbf{c}_{11} - \mathbf{c}_{12}, w)$ such that $\mathbf{r}_2 \mathbf{G}_{RS} + \bar{\mathbf{e}} = \mathbf{c}_{11} - \mathbf{c}_{12}$, where $|\bar{\mathbf{e}}| \leq w$
- 9: Compute $\mathbf{r}_1 = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1} (\mathbf{G}_U)_{I_1}^{-1}$, where $I_1 \subseteq I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$ and $|I_1| = k_1$
- 10: Compute $\mathbf{r} = (\mathbf{r}_1 \parallel \mathbf{r}_2) \mathbf{A}$ and $\mathbf{e} = \mathbf{c}_1 - \mathbf{r} \mathbf{G}_{\text{pk}}$. If $|\mathbf{e}| \leq w$, continue; otherwise, return to Step 9 and choose a new set I_1 to recompute $\mathbf{r}_1$

Output: $\mathbf{m} = \mathbf{c}_2 \oplus \mathcal{H}(\mathbf{r}, \mathbf{e})$

Correctness: $\mathbf{G}_{\text{pk}}$ is derived from $\mathbf{G}_{\text{sk}}$ via Gaussian elimination, so $\mathbf{G}_{\text{pk}} = (\mathbf{I}_k \parallel \mathbf{T}) = \mathbf{S} \mathbf{G}_{\text{sk}}$, where $\mathbf{S}$ is an invertible matrix representing the row operations. Let $\mathbf{G}_{\text{sk}} = (\mathbf{A} \parallel \mathbf{B})$, where $\mathbf{A}$ is the left $k \times k$ square matrix and $\mathbf{B}$ is the right $k \times (n - k)$ matrix. Then $\mathbf{S} \cdot (\mathbf{A} \parallel \mathbf{B}) = (\mathbf{I}_k \parallel \mathbf{T})$, implying $\mathbf{A} = \mathbf{S}^{-1}$. Given $\mathbf{r} \mathbf{S} \mathbf{G}_{\text{sk}} + \mathbf{e} = \mathbf{c}_1$, let $\mathbf{r} \mathbf{S} = (\mathbf{r}_1 \parallel \mathbf{r}_2)$, $\mathbf{e} \mathbf{D}^{-1} = (\mathbf{e}_1 \parallel \mathbf{e}_2)$, and $\mathbf{c}_1 \mathbf{D}^{-1} = (\mathbf{c}_{11} \parallel \mathbf{c}_{12})$. According to the definition of the private matrix $\mathbf{G}_{\text{sk}}$, we have

$$\mathbf{r}_1 \mathbf{G}_U + \mathbf{r}_2 (\mathbf{G}_{RS} + \mathbf{G}_W) + \mathbf{e}_1 = \mathbf{c}_{11}, \quad \mathbf{r}_1 \mathbf{G}_U + \mathbf{r}_2 \mathbf{G}_W + \mathbf{e}_2 = \mathbf{c}_{12}. \quad (13)$$

Subtracting the two equations yields

$$\mathbf{r}_2 \mathbf{G}_{RS} + \mathbf{e}_1 - \mathbf{e}_2 = \mathbf{c}_{11} - \mathbf{c}_{12},$$

where the vectors $\mathbf{r}_2$ and $\mathbf{e}_1 - \mathbf{e}_2$ are exactly what is computed in Step 8 of the decryption algorithm, and $\bar{\mathbf{e}} = \mathbf{e}_1 - \mathbf{e}_2$. After obtaining vectors $\mathbf{r}_2$ and $\bar{\mathbf{e}}$, using the right equation in (13) gives

$$\mathbf{r}_1 \mathbf{G}_U + \mathbf{e}_2 = \mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W.$$

Given $(\mathbf{e}_1 || \mathbf{e}_2) = \mathbf{e} \mathbf{D}^{-1}$, let $I_2 = \text{supp}(\mathbf{e}_2)$. If for all $i \in I_2$, $\mathbf{e}_2(i) \neq \mathbf{e}_1(i)$, then the inclusion $\text{supp}(\mathbf{e}_2) \subseteq \text{supp}(\bar{\mathbf{e}})$ holds. Using this inclusion, it is known that the coordinates of vector $\mathbf{e}_2$ indexed by the set $I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$ are certainly error-free, i.e., for any $i \in I$, $\mathbf{e}_2(i) = 0$. Therefore, as long as we pick any subset I_1 of size k_1 from $I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$, we have

$$\mathbf{r}_1(\mathbf{G}_U)_{I_1} + (\mathbf{e}_2)_{I_1} = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1},$$

i.e.,

$$\mathbf{r}_1 = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1} (\mathbf{G}_U)_{I_1}^{-1}.$$

After obtaining $\mathbf{r}_1$ and $\mathbf{r}_2$, vectors $\mathbf{r}$ and $\mathbf{e}$ can be easily computed, yielding

$$\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, \mathbf{m})) = \mathbf{m}. \quad (14)$$

If there exists $i \in I_2$ such that $\mathbf{e}_2(i) = \mathbf{e}_1(i)$, the inclusion $\text{supp}(\mathbf{e}_2) \subseteq \text{supp}(\bar{\mathbf{e}})$ does not hold. Hence, the subset I_1 chosen in Step 9 from $I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$ might, with some probability, contain indices of non-zero components of $\mathbf{e}_2$, i.e., $I_1 \cap \text{supp}(\mathbf{e}_2) \neq \emptyset$. In that case, the vector $\mathbf{e}$ computed in Step 10 will not satisfy $wt(\mathbf{e}) = w$. The algorithm then returns to Step 9, selects a new set $I_1 \subseteq I = [n/2] \setminus \text{supp}(\bar{\mathbf{e}})$, and recomputes until $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$. Then,

$$\mathbf{r}_1(\mathbf{G}_U)_{I_1} + (\mathbf{e}_2)_{I_1} = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1}, \quad (\mathbf{e}_2)_{i \in I_1}(i) = 0,$$

i.e.,

$$\mathbf{r}_1 = (\mathbf{c}_{12} - \mathbf{r}_2 \mathbf{G}_W)_{I_1} (\mathbf{G}_U)_{I_1}^{-1}.$$

After correctly obtaining $\mathbf{r}_1$ and $\mathbf{r}_2$, vectors $\mathbf{r}$ and $\mathbf{e}$ can be correctly computed, yielding

$$\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, \mathbf{m})) = \mathbf{m}. \quad (15)$$

In the UVW decryption algorithm, it is necessary to check whether the computed error vector $\mathbf{e}$ satisfies $wt(\mathbf{e}) = w$. This is because the set I_1 selected in UVW might, with some probability, contain indices of non-zero components of $\mathbf{e}_2$ ($I_1 \cap \text{supp}(\mathbf{e}_2) \neq \emptyset$), leading to decryption failure. The decryption failure probability is analyzed in detail in Section 2.2.

1.2.2 UVW Key Encapsulation Mechanism

In this section, we apply the Fujisaki-Okamoto transform [4] to the UVW public-key encryption scheme UVW.PKE to obtain the corresponding UVW key encapsulation mechanism UVW.KEM. A brief description of the KEM is as follows:

Assume Alice and Bob wish to share a session key. Bob first publishes his public key $\text{pk} = \mathbf{T}$ and keeps the corresponding private key $\text{sk} = (\text{seed}, i_1, i_2, i_3)$. Let $\mathcal{H}_1, \mathcal{H}_2, \mathcal{H}_3$ and $\mathcal{H}_4$ be four secure hash functions. We denote by $\text{UVW.PKE.Enc}(\text{pk}, \mathbf{m}; \sigma)$ the deterministic execution of the encryption algorithm $\text{UVW.PKE.Enc}(\text{pk}, \mathbf{m})$ using random seed σ .

Algorithm 6 UVW.KEM.Setup(1^λ): Initialization Algorithm

Input: Security parameter λ

- 1: Select system parameters $params = (q, n, k, k_1, k_2, w, m)$ according to the security parameter, where $k = k_1 + k_2$.
 - 2: Select four secure hash functions: $\mathcal{H}_1 : \{0, 1\}^* \rightarrow \mathbb{F}_q^k \times \mathbb{F}_q^n$, $\mathcal{H}_2 : \{0, 1\}^* \rightarrow \mathbb{F}_2^m$, $\mathcal{H}_3 : \{0, 1\}^* \rightarrow \mathbb{F}_2^m$, $\mathcal{H}_4 : \{0, 1\}^* \rightarrow \mathbb{F}_2^{512}$.
 - 3: Define the finite field $\mathbb{F}_q$; all subsequent operations are performed over $\mathbb{F}_q$.
-

Algorithm 7 UVW.KEM.KeyGen($params$): Key Generation Algorithm

Input: System parameters $params$

- 1: Call UVW.PKE's key generation algorithm UVW.PKE.KeyGen($params$) to generate the private key sk and public key pk .

Output: Private key $sk = (seed, i_1, i_2, i_3)$, public key $pk = \mathbf{T}$

Algorithm 8 UVW.KEM.Encap(pk): Encapsulation Algorithm

Input: Public key pk

- 1: Randomly select $\mathbf{m} \in \mathbb{F}_2^m$, compute $(\mathbf{r}, \mathbf{e}) = \mathcal{H}_1(\mathbf{m})$ and $\mathbf{d} = \mathcal{H}_2(\mathbf{m})$.
- 2: Compute $(\mathbf{c}_1, \mathbf{c}_2) = \text{UVW.PKE.Enc}(pk, \mathbf{m}, \mathbf{r}, \mathbf{e})$, where $\mathbf{c}_1 = \mathbf{r}(\mathbf{I}_k \parallel \mathbf{T}) + \mathbf{e}$, $\mathbf{c}_2 = \mathbf{m} \oplus \mathcal{H}_3(\mathbf{r}, \mathbf{e})$.
- 3: Set session key $K = \mathcal{H}_4(\mathbf{m}, \mathbf{c}_1, \mathbf{c}_2)$.

Output: $(\mathbf{c}_1, \mathbf{c}_2, \mathbf{d})$

Algorithm 9 UVW.KEM.Decap($sk, (\mathbf{c}_1, \mathbf{c}_2, \mathbf{d})$): Decapsulation Algorithm

Input: Private key sk , ciphertext $(\mathbf{c}_1, \mathbf{c}_2, \mathbf{d})$

- 1: Call UVW.PKE.Dec($sk, (\mathbf{c}_1, \mathbf{c}_2)$) to obtain $\mathbf{m}'$.
- 2: Compute $(\mathbf{r}', \mathbf{e}') = \mathcal{H}_1(\mathbf{m}')$ and $\mathbf{d}' = \mathcal{H}_2(\mathbf{m}')$.
- 3: Compute $(\mathbf{c}'_1, \mathbf{c}'_2) = \text{UVW.PKE.Enc}(pk, \mathbf{m}', \mathbf{r}', \mathbf{e}')$.
- 4: If $(\mathbf{c}_1, \mathbf{c}_2, \mathbf{d}) = (\mathbf{c}'_1, \mathbf{c}'_2, \mathbf{d}')$, continue; otherwise, abort.

Output: $K = \mathcal{H}_4(\mathbf{m}, \mathbf{c}_1, \mathbf{c}_2)$

1.3 Algorithm Instances and Parameters

Based on the optimal known attacks (decoding attacks and code distinguishing attacks), we selected three parameter sets for the UVW KEM, targeting classical security levels of 128 bits, 256 bits, and 512 bits, with quantum security levels of at least 80 bits, 128 bits, and 256 bits, respectively.

The hash functions employed in the scheme are instantiated via SM3-based extensible output function algorithms.

Table 1: Recommended Parameter Sets for the UVW KEM

Scheme	$(q, n, k, k_1, k_2, w, m)$	$ \mathbf{pk} $ (Bytes)
UVW128	(433, 860, 430, 215, 215, 116, 256)	208013
UVW256	(857, 1708, 854, 427, 427, 232, 256)	911645
UVW512	(1709, 3412, 1706, 853, 853, 463, 512)	4001850

2 Design Rationale

2.1 Underlying Hard Problems

The General Decoding (GD) problem is one of the most widely used problems in code-based cryptography, and the security of our UVW scheme also relies on it. The Syndrome Decoding (SD) problem is the dual of the General Decoding problem; in cryptanalysis, the two problems are computationally equivalent.

Problem 1 (General Decoding (GD) Problem). *Given a generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a random $[n, k]$ linear code $\mathcal{C}$ over $\mathbb{F}_q$, a vector $\mathbf{r} \in \mathbb{F}_q^n$, and a positive integer w , find a vector $\mathbf{m} \in \mathbb{F}_q^k$ such that $wt(\mathbf{m}\mathbf{G} - \mathbf{r}) \leq w$.*

Problem 2 (Syndrome Decoding (SD) Problem). *Given a parity-check matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$ of a random $[n, k]$ linear code $\mathcal{C}$ over $\mathbb{F}_q$, a vector $\mathbf{s} \in \mathbb{F}_q^{n-k}$, and a positive integer w , find a vector $\mathbf{e} \in \mathbb{F}_q^n$ such that $\mathbf{H}\mathbf{e}^T = \mathbf{s}^T$ and $wt(\mathbf{e}) \leq w$.*

The hardness of the GD problem is associated with random codes. Since we construct our cryptographic scheme using generalized $(U + V, U + W)$ codes, we must also consider the hardness of the Generalized $(U + V, U + W)$ Code Distinguishing (CD) problem.

Let $\mathcal{G}_{\mathbf{pk}}$ be the set of all possible public key matrices generated by the UVW.PKE scheme. A $\mathcal{G}_{\mathbf{pk}}$ distinguisher $\mathcal{D}$ is a probabilistic polynomial-time algorithm. Its advantage is defined as:

$$Adv^{CD}(\mathcal{D}) = \left| \mathbb{P}[\mathbf{G} \xleftarrow{\$} \mathcal{G}_{\mathbf{pk}} : \mathcal{D}(\mathbf{G}) = 1] - \mathbb{P}[\mathbf{G} \xleftarrow{\$} \mathbb{F}_q^{k \times n} : \mathcal{D}(\mathbf{G}) = 1] \right|. \quad (16)$$

If this advantage is negligible for any distinguisher $\mathcal{D}$, then the public key matrix of the UVW.PKE scheme is indistinguishable from a random matrix, meaning the generalized $(U + V, U + W)$ code is indistinguishable from a random linear code.

Definition 8. *Let $\mathcal{G}_{\mathbf{pk}}$ be the set of all possible public key matrices generated by the UVW.PKE scheme. If for any $\mathcal{G}_{\mathbf{pk}}$ distinguisher $\mathcal{D}$ running in time at most τ ,*

$$Adv^{CD}(\mathcal{D}) \leq \epsilon,$$

then the Generalized $(U + V, U + W)$ Code Distinguishing problem is (τ, ϵ) -hard.

2.2 Decryption Failure Probability Analysis

In Step 9 of the UVW PKE decryption algorithm, the decryptor needs to randomly select a subset I_1 of size k_1 from the inferred error-free coordinate index set $I = [n/2] \setminus \text{supp}(\bar{e})$. Due to the randomness of

this selection, if the chosen subset I_1 happens to contain a non-zero index of the error vector component e_2 (i.e., $I_1 \cap \text{supp}(e_2) \neq \emptyset$), the decryption algorithm will fail to correctly solve for an error vector satisfying $wt(e) = w$, causing a decryption failure. To accurately assess the reliability of the scheme, this failure introduced by random sampling must be quantified. Theorem 2 provides the probability that a single random selection of I_1 perfectly avoids the support of e_2 ($I_1 \cap \text{supp}(e_2) = \emptyset$), i.e., the single-decryption success probability.

Theorem 2. *Let I_1 be the set randomly selected in Step 9 of the UVW decryption algorithm. The probability that $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$ is*

$$\mathbb{P}(I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset) = \frac{1}{\binom{n}{w}(q-1)^w} \left(\sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \binom{\frac{n}{2}}{t} \binom{\frac{n-2t}{2}}{i} \binom{\frac{n-2t-2i}{2}}{\frac{w-2t-i}{2}} \right. \\ \left. 2^i (q-2)^{\frac{w-2t-i}{2}} (q-1)^{\frac{w+i}{2}} \frac{\binom{\frac{n-w-i}{2}}{k_1}}{\binom{\frac{n-w+2t-i}{2}}{k_1}} \right).$$

Proof. Let $\mathbf{e} = (\mathbf{e}_1 | \mathbf{e}_2)$ be a uniformly random vector of weight w from $\mathcal{E}_{q,n}^w$, where $\mathbf{e}_1, \mathbf{e}_2 \in \mathbb{F}_q^{n/2}$. Define three intermediate weights:

$$\begin{aligned} |\mathbf{e}|^1 &= |\{i \in [n/2] \mid \mathbf{e}_j(i) = 0, \mathbf{e}_{3-j}(i) \neq 0, j = 1, 2\}|, \\ |\mathbf{e}|^2 &= |\{i \in [n/2] \mid \mathbf{e}_1(i) \neq 0, \mathbf{e}_2(i) \neq 0, \mathbf{e}_1(i) \neq \mathbf{e}_2(i)\}|, \\ |\mathbf{e}|^3 &= |\{i \in [n/2] \mid \mathbf{e}_1(i) \neq 0, \mathbf{e}_2(i) \neq 0, \mathbf{e}_1(i) = \mathbf{e}_2(i)\}|. \end{aligned}$$

Then the following two equations hold:

$$|\mathbf{e}| = |\mathbf{e}|^1 + 2|\mathbf{e}|^2 + 2|\mathbf{e}|^3, \quad |\mathbf{e}_1 - \mathbf{e}_2| = |\mathbf{e}|^1 + |\mathbf{e}|^2.$$

We observe that if $|\mathbf{e}|^3 = 0$, then $\text{supp}(\mathbf{e}_2) \subseteq \text{supp}(\mathbf{e}_1 - \mathbf{e}_2)$, and the event $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$ always occurs. However, if $|\mathbf{e}|^3 \neq 0$, both events $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$ and $I_1 \cap \text{supp}(\mathbf{e}_2) \neq \emptyset$ are possible.

By the definitions of $|\mathbf{e}|^1$, $|\mathbf{e}|^2$, and $|\mathbf{e}|^3$, we have

$$\begin{aligned} \mathbb{P}(I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset) &= \sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \mathbb{P}(|\mathbf{e}|^3 = t, |\mathbf{e}|^1 = i, |\mathbf{e}|^2 = \frac{w-2t-i}{2}) \frac{\binom{\frac{n}{2}-i-\frac{w-2t-i}{2}-t}{k_1}}{\binom{\frac{n}{2}-i-\frac{w-2t-i}{2}}{k_1}} \\ &= \sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \mathbb{P}(|\mathbf{e}|^3 = t, |\mathbf{e}|^1 = i, |\mathbf{e}|^2 = \frac{w-2t-i}{2}) \frac{\binom{\frac{n-w-i}{2}}{k_1}}{\binom{\frac{n-w+2t-i}{2}}{k_1}} \\ &= \sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \left(\frac{1}{\binom{n}{w}(q-1)^w} \binom{n/2}{t} (q-1)^t \binom{n/2-t}{i} (2q-2)^i \right. \\ &\quad \left. \binom{n/2-t-i}{\frac{w-2t-i}{2}} ((q-2)(q-1))^{\frac{w-2t-i}{2}} \right) \frac{\binom{\frac{n-w-i}{2}}{k_1}}{\binom{\frac{n-w+2t-i}{2}}{k_1}} \\ &= \frac{1}{\binom{n}{w}(q-1)^w} \left(\sum_{t=0}^{\frac{w}{2}} \sum_{i=0}^{w-2t} \binom{n/2}{t} \binom{\frac{n-2t}{2}}{i} \binom{\frac{n-2t-2i}{2}}{\frac{w-2t-i}{2}} \right. \\ &\quad \left. 2^i (q-2)^{\frac{w-2t-i}{2}} (q-1)^{\frac{w+i}{2}} \frac{\binom{\frac{n-w-i}{2}}{k_1}}{\binom{\frac{n-w+2t-i}{2}}{k_1}} \right). \end{aligned}$$

This completes the proof of Theorem 2. $\square$

We emphasize that for the parameters selected in Section 1.3, the probability $I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$ is close to 1. Specifically, for UVW128, UVW256, and UVW512, the single-decryption success probabilities are 0.988123, 0.987827, and 0.987803, respectively.

2.3 Parameter Selection Rationale

To balance security and implementation performance, the selection of system parameters $\text{params} = (q, n, k, k_1, k_2, w, m)$ accounts for the following two aspects:

Reliability As described in Section 2.2, to ensure that the subset I_1 selected during decryption does not contain non-zero components of the error vector ($I_1 \cap \text{supp}(\mathbf{e}_2) = \emptyset$), the parameters q , n , k_1 , and w must guarantee a very high single-trial success probability. Through parameter optimization, the chosen sets ensure a success probability approaching 1, thereby guaranteeing the scheme's reliability.

Security The selected system parameters must be sufficiently resistant to the best-known cryptanalytic methods. We primarily considered general decoding attacks against random linear codes (ISD algorithms, quantum ISD algorithms) and distinguishing attacks against the generalized $(U+V, U+W)$ code structure. Details are provided in Section 3.2, "Concrete Security".

3 Security Analysis

3.1 Security Proof

Indistinguishability under Chosen-Plaintext Attack (IND-CPA) and Indistinguishability under Chosen-Ciphertext Attack (IND-CCA) are the two most common security models for public-key encryption schemes. In the IND-CPA model, an adversary can query ciphertexts of arbitrary messages; its goal is to distinguish which of two candidate plaintexts corresponds to a given challenge ciphertext.

Let $\Pi = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ be a public-key encryption scheme. The formal definition of IND-CPA security is given below.

Definition 9 (IND-CPA Security). *Let $\Pi = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ be a public-key encryption scheme, and λ a security parameter. If for any probabilistic polynomial-time adversary $\mathcal{A}$, the following experiment $\text{Exp}_{\mathcal{A}, \Pi}^{\text{IND-CPA}}(1^\lambda)$ outputs 1 with probability negligibly close to $1/2$, i.e.,*

$$|\mathbb{P}[\text{Exp}_{\mathcal{A}, \Pi}^{\text{IND-CPA}}(1^\lambda) = 1] - \frac{1}{2}| \leq \text{negl}(\lambda), \quad (17)$$

then the scheme Π is IND-CPA secure.

$\text{Exp}_{\mathcal{A}, \Pi}^{\text{IND-CPA}}(1^\lambda) :$

- 1) $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda);$
- 2) $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(\text{pk}, 1^\lambda)$ (i.e., $\mathcal{A}$ selects two messages of polynomial size in λ . In the PKE setting, the adversary possesses the public key, which is equivalent to having access to the encryption oracle; encryption queries are omitted here);

- 3) $b \xleftarrow{\$} \{0, 1\}$; the challenger $\mathcal{C}$ computes $\mathbf{c} \leftarrow \text{Encrypt}(\text{pk}, \mathbf{m}_b)$ and sends $\mathbf{c}$ to $\mathcal{A}$;
- 4) $b' \leftarrow \mathcal{A}(\text{pk}, 1^\lambda, \mathbf{c})$;
- 5) If $b' = b$, return 1; otherwise return 0.

Compared to IND-CPA, a stronger security model is IND-CCA, where the adversary is allowed access to a decryption oracle. IND-CCA is further subdivided into IND-CCA1 and IND-CCA2. In the IND-CCA1 model, the adversary can query the decryption oracle arbitrarily before selecting the challenge message pair $(\mathbf{m}_0, \mathbf{m}_1)$. In the IND-CCA2 model, the adversary can access the decryption oracle both before and after receiving the challenge ciphertext $\mathbf{c}$ (but cannot query the decryption of $\mathbf{c}$). An IND-CPA secure PKE scheme can be elevated to IND-CCA security via generic transforms, such as the Naor-Yung transform [5] or the Fujisaki-Okamoto transform [4].

Since our UVW.KEM is obtained by applying the Fujisaki-Okamoto transform to the UVW.PKE scheme, proving that UVW.PKE is IND-CPA secure implies that UVW.KEM is IND-CCA secure.

We now provide the security proof for the UVW.PKE scheme. Assuming the hardness of the decoding and code distinguishing problems, we prove that the PKE scheme is IND-CPA secure in the random oracle model.

Theorem 3. *Let t_{enc} be the time to perform one encryption, t_H the time to respond to one random oracle query, and q_H the maximum number of hash queries made by the adversary. Suppose the $\mathcal{G}_{\text{pk}}$ code distinguishing problem (CD) is $(\tau_{CD}, \epsilon_{CD})$ -hard, and the general decoding problem (GD) is $(\tau_{GD}, \epsilon_{GD})$ -hard. Then UVW.PKE is (τ', ϵ') -IND-CPA secure in the random oracle model, where $\epsilon' = 2(\epsilon_{CD} + \epsilon_{GD})$, and $\tau' \approx \min(\tau_{CD}, \tau_{GD}) - q_H \cdot t_H - t_{\text{enc}}$.*

Proof. We prove IND-CPA security by constructing a sequence of games, each computationally indistinguishable from the next, culminating in the indistinguishability of ciphertexts of $\mathbf{m}_0$ and $\mathbf{m}_1$. If an adversary $\mathcal{A}$ can distinguish two adjacent games, we can use $\mathcal{A}$ to construct an algorithm $\mathcal{B}$ that either distinguishes the public key matrix from a random matrix or solves the general decoding problem.

Let **Game 0** (Fig. 1) be the real IND-CPA game for encrypting $\mathbf{m}_0$. Let $\mathbb{P}(S_i)$ be the probability that $\mathcal{A}$ outputs $b' = 0$ in **Game** i . Then

$$\mathbb{P}(S_0) = \mathbb{P}[b' \leftarrow \mathcal{A}(\text{pk}, \lambda, (\mathbf{c}_1, \mathbf{c}_2)), b' = 0]. \quad (18)$$

```

1  $params \leftarrow \text{Setup}(1^\lambda)$ 
2  $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(params)$ 
3  $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(\text{pk}, \lambda)$ 
4  $(\mathbf{c}_1, \mathbf{c}_2) = (\mathbf{r}\mathbf{G}_{\text{pk}} + \mathbf{e}, \mathbf{m}_0 \oplus \text{Hash}(\mathbf{r}, \mathbf{e})) \leftarrow \text{Encrypt}(\text{pk}, \mathbf{m}_0)$ 
5  $b' \leftarrow \mathcal{A}(\text{pk}, \lambda, (\mathbf{c}_1, \mathbf{c}_2))$ 

```

Figure 1: **Game 0:** The real IND-CPA game for encrypting message

In **Game 0**, $\mathbf{r}$ is chosen uniformly from $\mathbb{F}_q^k$, and $\mathbf{e} \in \mathbb{F}_q^n$ is a uniformly chosen vector of length n , weight w , with distinct non-zero components.

```

1  $params \leftarrow \text{Setup}(1^\lambda)$ 
2  $(pk, sk) \leftarrow \text{KeyGen}(params)$ 
3  $pk \leftarrow \mathbf{G}$ 
4  $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(pk, \lambda)$ 
5  $(\mathbf{c}_1, \mathbf{c}_2) = (\mathbf{r}\mathbf{G} + \mathbf{e}, \mathbf{m}_0 \oplus \text{Hash}(\mathbf{r}, \mathbf{e})) \leftarrow \text{Encrypt}(pk, \mathbf{m}_0)$ 
6  $b' \leftarrow \mathcal{A}(pk, \lambda, (\mathbf{c}_1, \mathbf{c}_2))$ 

```

Figure 2: **Game 1**: replaces the public key matrix $\mathbf{G}_{pk}$ with a random matrix $\mathbf{G}$

Game 1. In this game, the challenger replaces the public key matrix $\mathbf{G}_{pk}$ with a random matrix $\mathbf{G}$ (Fig. 2). To relate **Game 0** and **Game 1**, we construct a distinguisher $\mathcal{D}$. If pk is generated by KeyGen , $\mathcal{D}$ plays **Game 0**, giving

$$\mathbb{P}[\mathbf{G}_{pk} \leftarrow \text{KeyGen}(params) : \mathcal{D}(\mathbf{G}_{pk}) = 1] = \mathbb{P}(S_0). \quad (19)$$

If pk is a random matrix, $\mathcal{D}$ plays **Game 1**, giving

$$\mathbb{P}[\mathbf{G} \xleftarrow{\$} \mathbb{F}_q^{k \times n} : \mathcal{D}(\mathbf{G}) = 1] = \mathbb{P}(S_1). \quad (20)$$

Thus, the advantage of $\mathcal{D}$ in distinguishing $\mathbf{G}_{pk}$ from random $\mathbf{G}$ is

$$\text{Adv}^{CD}(\mathcal{D}) = |\mathbb{P}(S_0) - \mathbb{P}(S_1)|. \quad (21)$$

Since the CD problem is $(\tau_{CD}, \epsilon_{CD})$ -hard, we have

$$|\mathbb{P}(S_0) - \mathbb{P}(S_1)| \leq \epsilon_{CD}. \quad (22)$$

```

1  $params \leftarrow \text{Setup}(1^\lambda)$ 
2  $(pk, sk) \leftarrow \text{KeyGen}(params)$ 
3  $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(pk, \lambda)$ 
4  $(\mathbf{c}_1, \mathbf{c}_2) = (\mathbf{r}\mathbf{G} + \mathbf{e}, \mathbf{m}_0 \oplus \mathbf{s}) \leftarrow \text{Encrypt}(pk, \mathbf{m}_0)$ 
5  $b' \leftarrow \mathcal{A}(pk, \lambda, (\mathbf{c}_1, \mathbf{c}_2))$ 

```

Figure 3: **Game 2**: replaces $\text{Hash}(\mathbf{r}, \mathbf{e})$ with a random bit string $\mathbf{s}$

Game 2. In this game, the challenger replaces $\text{Hash}(\mathbf{r}, \mathbf{e})$ with a random bit string $\mathbf{s}$ (Fig. 3). We first analyze $\mathbb{P}(S_2)$ in **Game 2**. From **Game 2**, the ciphertext components $\mathbf{c}_1$ and $\mathbf{c}_2$ are independent. Since the choices of $\mathbf{r}$, $\mathbf{G}$, and $\mathbf{e}$ are independent of $\mathbf{m}_0$, $\mathbf{c}_1$ is independent of $\mathbf{m}_0$. Moreover, $\mathbf{c}_2 = \mathbf{m}_0 \oplus \mathbf{s}$, where $\mathbf{s}$ is a random bit string, so $\mathbf{c}_2$ is also independent of $\mathbf{m}_0$. Consequently, the ciphertext $(\mathbf{c}_1, \mathbf{c}_2)$ is independent of $\mathbf{m}_0$, forcing the adversary to guess b' uniformly at random:

$$\mathbb{P}(S_2) = \frac{1}{2}. \quad (23)$$

To analyze the indistinguishability of **Game 1** and **Game 2**, we monitor the adversary's queries to the random oracle Hash . Let Query be the event that $\mathcal{A}$ queries the oracle on the vector pair $(\mathbf{r}, \mathbf{e})$ used to generate the challenge ciphertext.

1. If **Query** does not occur: under the random oracle model, the output $\text{Hash}(\mathbf{r}, \mathbf{e})$ is statistically indistinguishable from a random string s for unqueried inputs. **Game 1** and **Game 2** are identical for the adversary.
2. If **Query** occurs: this implies the adversary has found $(\mathbf{r}, \mathbf{e})$. We consider two cases:
 - Case A (Pre-challenge query): The adversary queries $(\mathbf{r}, \mathbf{e})$ before seeing the challenge ciphertext. Since $\mathbf{r}, \mathbf{e}$ are chosen uniformly at random in the challenge phase and are independent of the adversary's view, this occurs with probability $\leq q_H/|\mathcal{S}|$, which is negligible ($|\mathcal{S}|$ is the size of the input space for the hash).
 - Case B (Post-challenge query): The adversary queries $(\mathbf{r}, \mathbf{e})$ after receiving $\mathbf{c}_1 = \mathbf{r}\mathbf{G} + \mathbf{e}$. Since $\mathbf{G}$ is a random matrix, this means the adversary solved the general decoding problem for the random linear code.

Hence, the difference between **Game 1** and **Game 2** is bounded by $\mathbb{P}(\text{Query})$, which is dominated by Case B (the hardness of GD):

$$|\mathbb{P}(S_1) - \mathbb{P}(S_2)| \leq \mathbb{P}(\text{Query}) \leq \epsilon_{GD}. \quad (24)$$

From Eqs. (22), (23), and (24):

$$|\mathbb{P}(S_0) - \frac{1}{2}| \leq \epsilon_{GD} + \epsilon_{CD}. \quad (25)$$

Now, starting from this game, the challenger encrypts the other message $\mathbf{m}_1$. Let **Game 3** (Fig. 4) be the real IND-CPA game for encrypting $\mathbf{m}_1$. Similarly, let $\mathbb{P}(S_i)$ be the probability that $\mathcal{A}$ outputs $b' = 0$ in **Game i**. Then

$$\mathbb{P}(S_3) = \mathbb{P}[b' \leftarrow \mathcal{A}(\text{pk}, \lambda, (\mathbf{c}_1, \mathbf{c}_2)), b' = 0]. \quad (26)$$

```

1  $params \leftarrow \text{Setup}(1^\lambda)$ 
2  $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(params)$ 
3  $(\mathbf{m}_0, \mathbf{m}_1) \leftarrow \mathcal{A}(\text{pk}, \lambda)$ 
4  $(\mathbf{c}_1, \mathbf{c}_2) = (\mathbf{r}\mathbf{G}_{\text{pk}} + \mathbf{e}, \mathbf{m}_1 \oplus \text{Hash}(\mathbf{r}, \mathbf{e})) \leftarrow \text{Encrypt}(\text{pk}, \mathbf{m}_1)$ 
5  $b' \leftarrow \mathcal{A}(\text{pk}, \lambda, (\mathbf{c}_1, \mathbf{c}_2))$ 

```

Figure 4: **Game 3**: The real IND-CPA game for encrypting message $\mathbf{m}_1$.

Game 3 differs from **Game 0** only in that the message is $\mathbf{m}_1$ instead of $\mathbf{m}_0$.

Game 4. In this game, the challenger replaces $\mathbf{G}_{\text{pk}}$ with a random matrix $\mathbf{G}$. The relationship between **Game 3** and **Game 4** is identical to that between **Game 0** and **Game 1**:

$$|\mathbb{P}(S_3) - \mathbb{P}(S_4)| \leq \epsilon_{CD}. \quad (27)$$

Game 5. In this game, the challenger replaces $\text{Hash}(\mathbf{r}, \mathbf{e})$ with a random string s . Similarly to **Game 2**, the adversary can only guess randomly in **Game 5**:

$$\mathbb{P}(S_5) = \frac{1}{2}. \quad (28)$$

Moreover, the relationship between **Game 4** and **Game 5** is identical to that between **Game 1** and **Game 2**:

$$|\mathbb{P}(S_4) - \mathbb{P}(S_5)| \leq \epsilon_{GD}. \quad (29)$$

From Eqs. (27), (28), and (29):

$$|\mathbb{P}(S_3) - \frac{1}{2}| \leq \epsilon_{GD} + \epsilon_{CD}. \quad (30)$$

Finally, combining (25) and (30):

$$|\mathbb{P}(S_0) - \mathbb{P}(S_3)| \leq 2(\epsilon_{GD} + \epsilon_{CD}), \quad (31)$$

i.e.,

$$\begin{aligned} Adv_{\mathcal{A}}^{\text{IND-CPA}} &= |\mathbb{P}[b' \leftarrow \mathcal{A}(\text{pk}, \lambda, \text{Encrypt}(\text{pk}, \mathbf{m}_0)), b' = 0] \\ &\quad - \mathbb{P}[b' \leftarrow \mathcal{A}(\text{pk}, \lambda, \text{Encrypt}(\text{pk}, \mathbf{m}_1)), b' = 0]| \leq 2(\epsilon_{GD} + \epsilon_{CD}). \end{aligned} \quad (32)$$

Thus, $\epsilon' = 2(\epsilon_{CD} + \epsilon_{GD})$.

Regarding runtime τ' , the reduction calls adversary $\mathcal{A}$ to build algorithm $\mathcal{B}$ for solving CD or GD. $\mathcal{B}$ runs $\mathcal{A}$, simulates the encryption oracle (costing t_{enc}), and responds to up to q_H hash queries (costing $q_H \cdot t_H$). Therefore, the effective running time τ' is bounded by the time to solve the underlying hard problem minus the simulation overhead:

$$\tau' \approx \min(\tau_{CD}, \tau_{GD}) - (q_H \cdot t_H + t_{enc}). \quad (33)$$

□

3.2 Concrete Security

3.2.1 Classical Security Analysis

The security of UVW relies on two hardness assumptions: 1) the decoding problem for random linear codes (Problem 1); 2) the problem of distinguishing whether a given linear code is a generalized $(U + V, U + W)$ code.

The first assumption the decoding problem is well known to be NP-complete; with appropriately chosen parameters, even quantum computers would require exponential time to solve it. The second assumption ensures that the public key matrix $\mathbf{G}_{\text{pk}} = \mathbf{S}\mathbf{G}_{\text{sk}}\mathbf{D}$ is indistinguishable from a random matrix. Structurally, $(U + V, U + W)$ codes look very similar to random codes. Only very subtle differences exist between random codes and $(U + V, U + W)$ codes of the same length and dimension, and these occur only among low-weight and high-weight codewords. When $\mathbf{v} = \mathbf{w} = \mathbf{0}$, codewords take the form $(\mathbf{u}, \mathbf{u})\mathbf{D}$; when $\mathbf{u} = \mathbf{0}$, they take the form $(\mathbf{v}, \mathbf{w})\mathbf{D}$. Both cases could potentially be exploited to distinguish whether a given linear code is a permuted $(U + V, U + W)$ code. Thus, the second hardness assumption essentially reduces to the problem of finding minimum-weight or maximum-weight codewords, which is fundamentally the same as the first assumption. Proposition 4 provides the weight distribution of a Reed-Solomon $[n, k]$ code over $\mathbb{F}_q$.

Proposition 4. *Let $\mathcal{C}$ be an $[n, k]$ Reed-Solomon code over $\mathbb{F}_q$. Let $a_1 = q-1$, $a_i = q^i - 1 - \sum_{j=1}^{i-1} \binom{n-k+i}{n-k+j} a_j$ ($i = 2, \dots, k$). Let N_i be the number of codewords in $\mathcal{C}$ of weight $n - k + i$. Then*

$$N_i = \binom{n}{n-k+i} a_i. \quad (34)$$

Proof. Reed-Solomon codes are Maximum Distance Separable (MDS); therefore, for an $[n, k]$ RS code $\mathcal{C}$, the codeword weights range in $[n - k + 1, n]$. The dual code $\mathcal{C}^\perp$ is an $[n, n - k]$ RS code. By definition of the dual code, $\mathbf{x} \in \mathcal{C}$ if and only if

$$\mathbf{x}\mathbf{H}^\top = \mathbf{0}, \quad (35)$$

where $\mathbf{H}$ is a generator matrix of $\mathcal{C}^\perp$. Hence, finding the weight distribution of RS code $\mathcal{C}$ is equivalent to finding the weight distribution of solutions to the linear system (35).

Since $\mathbf{H}$ is a generator matrix of an $[n, n - k]$ RS code, any $n - k$ columns of $\mathbf{H}$ form a Vandermonde matrix, which has non-zero determinant (i.e., the $n - k$ columns are linearly independent). Therefore, system (35) has no solution of weight $n - k$. On the other hand, any $n - k + 1$ columns are guaranteed to be linearly dependent, yielding non-zero solutions. This corresponds to a linear system with $n - k$ equations and $n - k + 1$ unknowns, providing 1 degree of freedom. Thus, system (35) has solutions of weight $n - k + 1$, and the number of such solutions is

$$\binom{n}{n - k + 1}(q - 1). \quad (36)$$

Next, we determine the number of solutions of weight $n - k + 2$. Selecting any $n - k + 2$ columns from $\mathbf{H}$ gives a system of $n - k$ equations in $n - k + 2$ unknowns. This system has 2 degrees of freedom, so the number of non-zero solutions is

$$q^2 - 1. \quad (37)$$

Subtracting those of weight $n - k + 1$, the number of weight- $n - k + 2$ solutions in this specific system is

$$q^2 - 1 - \binom{n - k + 2}{n - k + 1}(q - 1). \quad (38)$$

Thus, the total number of weight- $n - k + 2$ solutions in (35) is

$$\binom{n}{n - k + 2} \left(q^2 - 1 - \binom{n - k + 2}{n - k + 1}(q - 1) \right). \quad (39)$$

Let

$$a_1 = q - 1, \quad a_2 = q^2 - 1 - \binom{n - k + 2}{n - k + 1}(q - 1). \quad (40)$$

Next, we find the number of solutions of weight $n - k + 3$.

Similarly, any $n - k + 3$ columns form a system of $n - k$ equations in $n - k + 3$ unknowns with 3 free variables. The number of non-zero solutions is $q^3 - 1$. Subtracting solutions of weights $n - k + 1$ and $n - k + 2$, the number of weight- $n - k + 3$ solutions in this system is

$$\begin{aligned} q^3 - 1 - \binom{n - k + 3}{n - k + 1}(q - 1) - \binom{n - k + 3}{n - k + 2} \left(q^2 - 1 - \binom{n - k + 2}{n - k + 1}(q - 1) \right) \\ = q^3 - 1 - \sum_{j=1}^2 \binom{n - k + 3}{n - k + j} a_j. \end{aligned} \quad (41)$$

Let $a_3 = q^3 - 1 - \sum_{j=1}^2 \binom{n - k + 3}{n - k + j} a_j$, then the number of weight- $n - k + 3$ solutions in (35) is $\binom{n}{n - k + 3} a_3$.

From the analysis, selecting any $n - k + i$ columns yields a system of $n - k$ equations in $n - k + i$ unknowns. The number of weight- $n - k + i$ solutions is the total number of non-zero solutions $q^i - 1$

minus the solutions of weights $n - k + 1, \dots, n - k + i - 1$. Therefore, the number of weight- $n - k + i$ solutions in (35) is

$$\binom{n}{n - k + i} a_i, \quad (42)$$

where

$$a_i = q^i - 1 - \sum_{j=1}^{i-1} \binom{n - k + i}{n - k + j} a_j. \quad (43)$$

□

Having the codeword weight distribution of an RS code, we now give the statistical distance between the weight distributions of an RS code and a random linear code of the same size.

Proposition 5. *Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be an $[n, k]$ Reed-Solomon code and an $[n, k]$ random linear code over $\mathbb{F}_q$, respectively. The statistical distance between their codeword weight distributions is*

$$\rho(|\mathcal{C}_1|, |\mathcal{C}_2|) = \frac{1}{2} \left(\sum_{t=1}^{n-k} \frac{\binom{n}{t} (q-1)^t}{q^n} + \sum_{t=n-k+1}^n \left| \frac{\binom{n}{t} (q-1)^t}{q^n} - \frac{N_{t-n+k}}{q^k} \right| \right). \quad (44)$$

Proof. According to Proposition 4, the weight distribution of $\mathcal{C}_1$ splits into two parts: 1) For $t \in [n - k]$, the proportion of codewords of weight t in $\mathcal{C}_1$ is 0. 2) For $t \in [n - k + 1, n]$, the proportion of codewords of weight t is $\frac{N_{t-n+k}}{q^k}$.

For a random linear code $\mathcal{C}_2$ over $\mathbb{F}_q$, the average proportion of codewords of weight t is $\frac{\binom{n}{t} (q-1)^t}{q^n}$.

By the definition of statistical distance,

$$\rho(|\mathcal{C}_1|, |\mathcal{C}_2|) = \frac{1}{2} \left(\sum_{t=1}^{n-k} \frac{\binom{n}{t} (q-1)^t}{q^n} + \sum_{t=n-k+1}^n \left| \frac{\binom{n}{t} (q-1)^t}{q^n} - \frac{N_{t-n+k}}{q^k} \right| \right).$$

□

We computed the statistical distance for two parameter sets: for $(q, n, k) = (503, 500, 300)$, $\rho(|\mathcal{C}_1|, |\mathcal{C}_2|) \approx 2^{-1913.4}$; for $(q, n, k) = (251, 250, 125)$, $\rho(|\mathcal{C}_1|, |\mathcal{C}_2|) \approx 2^{-627.3}$. This shows that with suitably chosen parameters, the weight distribution of an RS code is indistinguishable from that of a random code. Furthermore, we obtain the following corollary.

Corollary 6. *Let $\mathbf{G}_1$ be a generator matrix of an $[n, k]$ RS code $\mathcal{C}_1$, and $\mathbf{G}_2, \mathbf{G}_3$ be generator matrices of two $[n, k]$ random linear codes $\mathcal{C}_2, \mathcal{C}_3$. Let $\mathcal{C}'$ be the linear code defined by generator $(\mathbf{G}_1 + \mathbf{G}_3 || \mathbf{G}_3)$, and $\mathcal{C}''$ by $(\mathbf{G}_2 + \mathbf{G}_3 || \mathbf{G}_3)$. Then the statistical distance between the weight distributions of $\mathcal{C}'$ and $\mathcal{C}''$ is*

$$\rho(|\mathcal{C}'|, |\mathcal{C}''|) = \rho(|\mathcal{C}_1|, |\mathcal{C}_2|) = \frac{1}{2} \left(\sum_{t=1}^{n-k} \frac{\binom{n}{t} (q-1)^t}{q^n} + \sum_{t=n-k+1}^n \left| \frac{\binom{n}{t} (q-1)^t}{q^n} - \frac{N_{t-n+k}}{q^k} \right| \right). \quad (45)$$

Since $\mathcal{C}''$ is a random code, the statistical distance between $\mathcal{C}'$ and a random code of the same length and dimension is the same. Therefore, when parameters are chosen appropriately, the weight distribution of the $(U + V, U + W)$ code in UVW (where $\mathbf{G}_V - \mathbf{G}_W$ is an RS generator matrix) can be treated as that of a $(U + V, U + W)$ code formed by three random matrices. Hence, finding codewords of the form $(\mathbf{v}, \mathbf{w})\mathbf{D}$ cannot be used to distinguish the generalized $(U + V, U + W)$ code.

Now, we analyze whether finding codewords of the form $(\mathbf{u}, \mathbf{u})\mathbf{D}$ can distinguish the generalized $(U + V, U + W)$ code. We treat $\mathbf{G}_U, \mathbf{G}_V, \mathbf{G}_W$ as random matrices when analyzing the weight distribution of the code they define. Proposition 7 gives the weight distribution of a generalized $[n, k]$ $(U + V, U + W)$ code over $\mathbb{F}_q$.

Proposition 7. *Let $\mathcal{C}$ be a generalized $[n, k]$ $(U + V, U + W)$ code over $\mathbb{F}_q$, where U is a random $[\frac{n}{2}, k_1]$ linear code and V, W are random $[\frac{n}{2}, k_2]$ linear codes. Let $N_{(\mathbf{u}, \mathbf{u})}$ and $N_{(\mathbf{u} + \mathbf{v}, \mathbf{u} + \mathbf{w})}$ be the average numbers of codewords of the forms $(\mathbf{u}, \mathbf{u})\mathbf{D}$ and $(\mathbf{u} + \mathbf{v}, \mathbf{u} + \mathbf{w})\mathbf{D}$, respectively. If w is even,*

$$N_{(\mathbf{u}, \mathbf{u})} = \frac{\binom{n/2}{w/2}(q-1)^{w/2}}{q^{n/2-k_1}}, \quad N_{(\mathbf{u} + \mathbf{v}, \mathbf{u} + \mathbf{w})} = N_{(\mathbf{u}, \mathbf{u})} + \frac{\binom{n}{w}(q-1)^w}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}}\right); \quad (46)$$

if w is odd,

$$N_{(\mathbf{u}, \mathbf{u})} = 0, \quad N_{(\mathbf{u} + \mathbf{v}, \mathbf{u} + \mathbf{w})} = \frac{\binom{n}{w}(q-1)^w}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}}\right). \quad (47)$$

For an adversary, distinguishing the generalized $(U + V, U + W)$ code from a random code amounts to finding a codeword of the form $(\mathbf{u}, \mathbf{u})\mathbf{D}$.

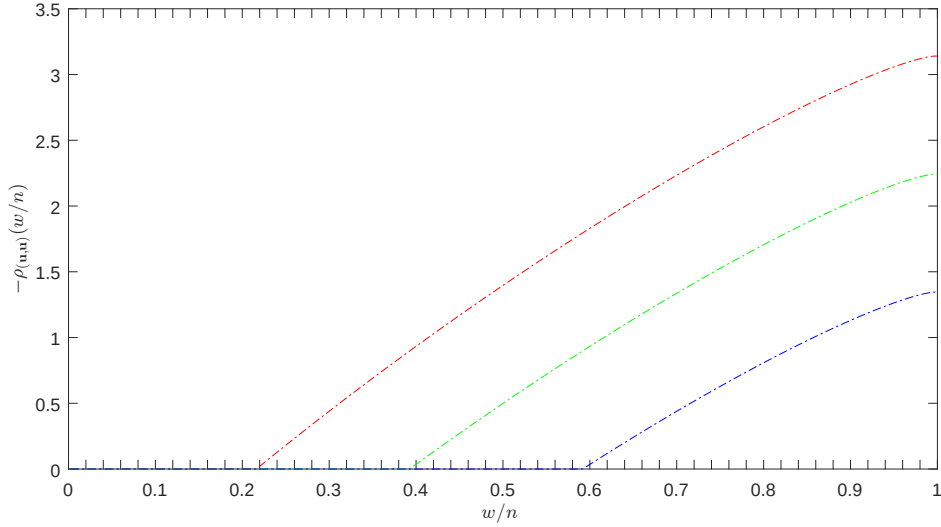


Figure 5: Normalized logarithm of the density of $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords as a function of relative weight w/n .

Figure 5 plots the normalized log-density of $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords for relative weight $x = \frac{w}{n}$, defined as

$$\rho_{(\mathbf{u}, \mathbf{u})}\left(\frac{w}{n}\right) = \frac{\log_2(N_{(\mathbf{u}, \mathbf{u})}/N_{(\mathbf{u} + \mathbf{v}, \mathbf{u} + \mathbf{w})})}{n}.$$

Using $q = 503, n = 1000, k = 500$, the three curves correspond to different values of k_1 and k_2 : from left to right, $k_1 = 0.3k, 0.5k, 0.7k$. We see that when the relative weight w/n is below 0.22, 0.4, and 0.59 respectively, almost all codewords are of the form $(\mathbf{u}, \mathbf{u})\mathbf{D}$. For $(q, n, k) = (503, 1000, 500)$, finding a codeword with weight in $[\frac{q-1}{q}(n-k) + 1, k + \frac{q-1}{q}(n-k)] = [500.006, 999.006]$ is easy. Thus, for $k_1 = 0.7k$, an adversary can easily distinguish the code, while for $k_1 = 0.3k, 0.5k$, the adversary requires exponential time.

We have proved that if the CD and GD problems are hard, UVW.PKE is IND-CPA secure. Since the CD problem ultimately reduces to finding a codeword of a specific weight, we can use information set decoding (ISD) algorithms to derive conservative security estimates.

ISD was first introduced by Prange [6] in 1962. Its concept involves identifying an information set and guessing that its coordinates are error-free. In 1988, Lee and Brickell [7] improved Prange's ISD by allowing a few errors in the information set. Later that year, Leon [8] and Stern [9] further improved upon Lee and Brickell's algorithm. Subsequently, almost all enhanced algorithms combined the meet-in-the-middle strategies of Lee-Brickell and Stern. Many improvements and analyses of ISD exist [10, 11, 12, 13, 14, 15].

In 2010, Peters [16] generalized the ISD algorithms of Lee-Brickell and Stern to arbitrary finite fields $\mathbb{F}_q$, producing the first q -ary ISD algorithm, though its complexity depends on q and degrades for larger q . In 2017, Niebuhr et al. [17] adopted ideas from Finiasz-Sendrier [18] to further optimize Peters' algorithm and derived lower bounds for the complexity of ISD over $\mathbb{F}_q$. This algorithm represents an idealized ISD variant, considering only essential steps and ignoring overhead/memory access costs, and is commonly used for conservative parameter estimation in code-based schemes. We do not detail the algorithm here; interested readers are referred to the literature. Below we give the expected work factor for finding a weight- w codeword using the algorithm from [17]:

Proposition 8. *Let $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$ be a random matrix and $\mathbf{s} \in \mathbb{F}_q^{n-k}$ a random syndrome. If $\binom{n}{w}(q-1)^w \geq q^{n-k}$ (multiple solutions) and $\binom{n-k}{w-v}\binom{k}{v}(q-1)^w \ll q^{n-k}$, the average decoding complexity for solving the SD instance $SD(\mathbf{H}, \mathbf{s}, w)$ using the algorithm from [17] is*

$$WF_{qISD}(q, n, k, w) = \min_{l, v_1, v_2} \frac{1}{\sqrt{q-1}} \frac{q^{n-k}}{\binom{n-k-l}{w-v}\binom{k}{v_1}\binom{l}{v_2}(q-1)^w} \left(K_q \frac{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}}{q^l} + \lambda_q^{-1} \left(\frac{2(q-1)l}{q\binom{l}{v_2'}(q-1)^{v_2'}} + v_2 \right) \sqrt{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}} \right), \quad (48)$$

where

$$v = v_1 + v_2, \quad v_2' = \lfloor v_2/2 \rfloor, \quad \lambda_q = 1 - \exp(-1) \approx 0.63, \quad K_q = 2w. \quad (49)$$

Here l, v_1, v_2 are optimization parameters.

If $\mathbf{s} = \mathbf{eH}^\top$ with $wt(\mathbf{e}) = w$, $\binom{n}{w}(q-1)^w < q^{n-k}$ (unique solution), and $\binom{n-k}{w-v}\binom{k}{v}(q-1)^w \ll q^{n-k}$, the average decoding complexity for $SD(\mathbf{H}, \mathbf{s}, w)$ is

$$WF_{qISD}(q, n, k, w) = \min_{l, v_1, v_2} \frac{1}{\sqrt{q-1}} \frac{\binom{n}{w}(q-1)^w}{\binom{n-k-l}{w-v}\binom{k}{v_1}\binom{l}{v_2}(q-1)^w} \left(K_q \frac{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}}{q^l} + \lambda_q^{-1} \left(\frac{2(q-1)l}{q\binom{l}{v_2'}(q-1)^{v_2'}} + v_2 \right) \sqrt{\binom{k}{v_1}\binom{l}{v_2}(q-1)^{v-1}} \right). \quad (50)$$

Example 9. For the UVW128 parameter set, $(q, n, k, w) = (433, 860, 430, 116)$. Since $\binom{n}{w}(q-1)^w \ll q^{n-k}$ (unique solution), we compute

$$WF_{qISD}(q, n, k, w) \approx 2^{128.3613}. \quad (51)$$

From Example 9, UVW128 achieves a classical security level of 128 bits against ISD attacks. Analogous analyses for UVW256 and UVW512 yield attack complexities of $2^{256.8354}$ and $2^{513.2195}$, confirming that these parameter sets also meet their target security levels.

We previously proved that the dual $\mathcal{C}^\perp$ of an $[n, k]$ ($U + V, U + W$) code $\mathcal{C}$ with parameters (k_1, k_2) is an $[n, n - k]$ ($U + V, U + W$) code with parameters $(n/2 - k_2, n/2 - k_1)$. Thus, the dimension of $(\mathbf{u}, \mathbf{u})\mathbf{D}$ -type codewords in $\mathcal{C}$ is k_1 , and in $\mathcal{C}^\perp$ it is $n/2 - k_2$. If $k_1 \neq n/2 - k_2$, both $\mathcal{C}$ and its dual must be considered for the distinguishing problem. If $k_1 = n/2 - k_2$, the distinguishing problems for $\mathcal{C}$ and $\mathcal{C}^\perp$ are equivalent. For simplicity, we choose parameters satisfying $k_1 = n/2 - k_2$, so only one distinguishing complexity needs analysis.

From Proposition 7, as w increases, the number of $(\mathbf{u}, \mathbf{u})\mathbf{D}$ -type codewords grows, but their proportion decreases. For ISD, more solutions mean lower complexity. We thus take the weight

$$t_{\max} = \max_w \left\{ w \mid \frac{\binom{n/2}{w/2}(q-1)^{w/2}}{q^{n/2-k_1}} > \frac{\binom{n}{w}(q-1)^w}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}}\right) \right\} \quad (52)$$

as the weight parameter for evaluating the distinguishing complexity.

On the other hand, to seek a codeword of weight $t_{\max}$, such a codeword must exist. Let

$$t_{\min} = \min_w \left\{ w \mid \frac{\binom{n/2}{w/2}(q-1)^{w/2}}{q^{n/2-k_1}} \geq 1 \right\}. \quad (53)$$

If $t_{\max} \geq t_{\min}$, codewords of weight $t_{\max}$ exist.

$$\text{Define } r_{t_{\min}} = \frac{\binom{n/2}{t_{\min}/2}(q-1)^{t_{\min}/2}}{q^{n/2-k_1}} / \frac{\binom{n}{t_{\min}}(q-1)^{t_{\min}}}{q^{n-k}} \leq \frac{q^{n/2-k_2}}{\binom{n/2}{t_{\min}/2}(q-1)^{t_{\min}/2}}.$$

If $k_1 \leq k_2$, then $r_{t_{\min}} \leq \frac{q^{n/2-k_1}}{\binom{n/2}{t_{\min}/2}(q-1)^{t_{\min}/2}} \leq 1$, implying $t_{\max} \leq t_{\min}$. That is, for all $t \geq t_{\min}$,

$$\frac{\binom{n/2}{t/2}(q-1)^{t/2}}{q^{n/2-k_1}} \leq \frac{\binom{n}{t}(q-1)^t}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}}\right). \quad (54)$$

Thus, when $k_1 \leq k_2$, the expected number of $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords is less than that of non- $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords.

Let E_1 be the expected number of $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords at weight t , and E_2 the expected number of non- $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords:

$$E_1 = \frac{\binom{n/2}{t/2}(q-1)^{t/2}}{q^{n/2-k_1}}, \quad E_2 = \frac{\binom{n}{t}(q-1)^t}{q^{n-k}} \left(1 - \frac{1}{q^{k_2}}\right). \quad (55)$$

We analyze E_1 and E_2 for UVW128.

From Fig. 6, when $k_1 = k_2$, $(\mathbf{u}, \mathbf{u})\mathbf{D}$ and non- $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords appear nearly simultaneously, and their expected counts satisfy Eq. (54). Their difference grows with t . For UVW128, finding a codeword of weight in $[430, 859]$ is easy, but at $t = 430$, E_2 is about 2^{428} times E_1 , so the probability of finding a $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codeword for $t \geq 430$ is negligible. Since code distinguishing relies on finding $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codewords, we analyze the hardness when expected counts are relatively close. Fig. 7 examines $t \in [330, 430]$. At $t = 336$, both types first appear and are closest in expected count; there, ISD complexity for any codeword is $\approx 2^{502.6478}$. At $t = 430$, ISD is easy, but $E_1 \ll E_2$, making it infeasible to find a $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codeword.

Let $F(t)$ be the ISD complexity to find any codeword of weight t , and $P(t) \approx \frac{E_1}{E_1 + E_2}$ the probability that a found codeword is of the $(\mathbf{u}, \mathbf{u})\mathbf{D}$ form. The complexity to find a $(\mathbf{u}, \mathbf{u})\mathbf{D}$ codeword is $F_{(\mathbf{u}, \mathbf{u})\mathbf{D}}(t) = F(t)/P(t)$, which measures the distinguishing hardness. For $t \in [336, 430]$, the minimum of $F_{(\mathbf{u}, \mathbf{u})\mathbf{D}}(t)$ is $2^{403.71}$ (at $t = 410$), far exceeding the 128-bit security threshold. For $t > 430$, the exponential growth

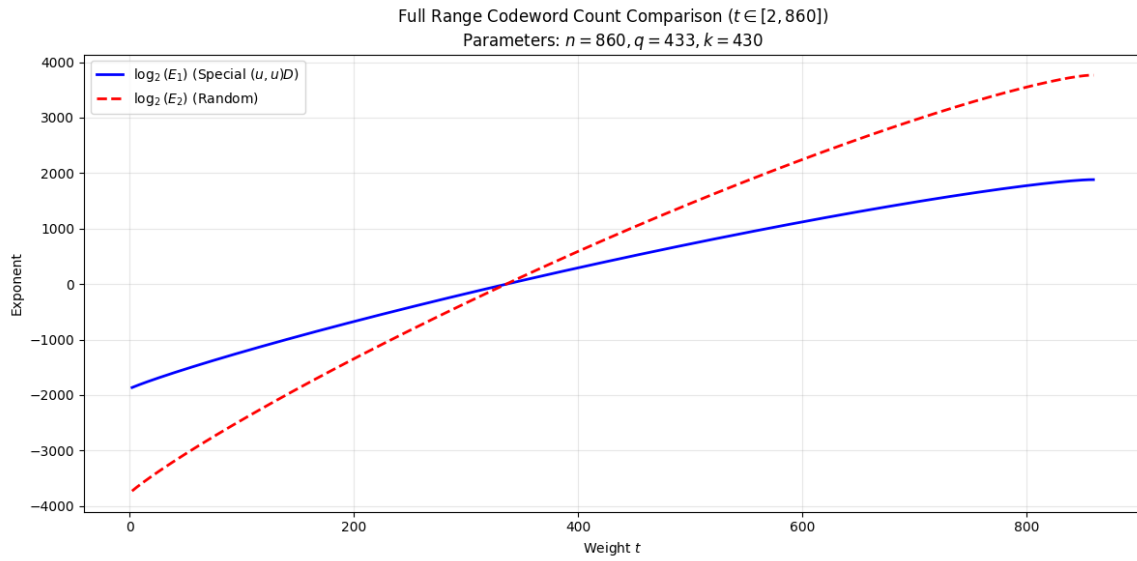


Figure 6: Comparison of expected numbers of $(u, u)D$ and non- $(u, u)D$ codewords as a function of weight t for UVW128.

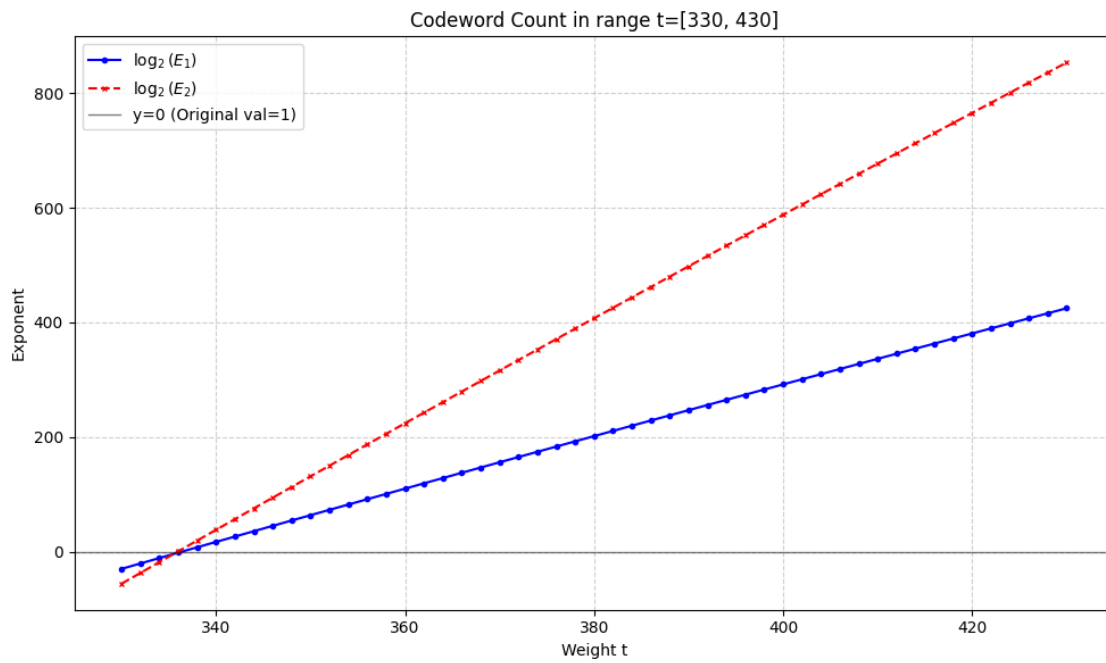


Figure 7: Comparison for weights $t \in [330, 430]$ for UVW128.

in expected count difference dominates, so complexity is higher. Thus UVW128 is secure. By similar analysis, the minimum $F_{(\mathbf{u}, \mathbf{u})\mathbf{D}}(t)$ for UVW256 and UVW512 are $2^{815.35}$ (at $t = 826$) and $2^{1649.78}$ (at $t = 1666$), surpassing the respective security levels.

3.2.2 Quantum Security Analysis

To evaluate the security of UVW against quantum attacks, we adopt the conservative quantum complexity estimation framework of Esser and Bellini [19]. This framework analyzes Grover-accelerated Prange-ISD under the NIST quantum circuit depth constraint (`maxdepth`) and provides quantum gate complexity estimates for the Syndrome Decoding (SD) problem. The best known generic attacks against the SD problem for random linear codes remain within the ISD family, where the main exponential complexity stems from searching for a suitable information set (or equivalently, a permutation). In the quantum setting, this search can be quadratically accelerated via Grover's algorithm.

It should be noted that the quantum complexity estimates in [19] primarily target the SD problem over binary fields $\mathbb{F}_2$. However, the core idea of the framework treating the permutation (information set) search in Prange-ISD as an unstructured search and applying Grover's algorithm with partitioning under the NIST depth constraint depends only on the success probability of a single attempt and the depth of the verification circuit. Hence, it naturally extends to the $\mathbb{F}_q$ setting.

In this work, we apply this estimation method to the SD problem for q -ary linear codes corresponding to UVW. We use the same combinatorial probability as in the binary case to approximate the success probability of information set selection and adopt the optimistic Gaussian elimination depth estimate from [19]. Since quantum circuit depths for linear algebra operations over $\mathbb{F}_q$ are typically no smaller than over $\mathbb{F}_2$, this approach represents an optimistic assumption for the attacker and yields a conservative estimate of the scheme's quantum security.

In Prange-ISD, the success probability of a single random permutation depends mainly on whether the error support falls within the redundancy coordinates. Following [19], we use the combinatorial probability from the binary case as an approximation:

$$p_{\text{succ}} := \frac{\binom{n-k}{w}}{\binom{n}{w}}. \quad (56)$$

Let D_E denote the quantum circuit depth required for Gaussian elimination and verification in one ISD attempt. According to [19],

$$D_E := (n - k)^{\omega_M}, \quad (57)$$

where $\omega_M \in [2, 3]$ is the matrix multiplication exponent. Consistent with [19], we take $\omega_M = 2.5$. (Note that the actual circuit depth for elimination and arithmetic over $\mathbb{F}_q$ is typically larger than over $\mathbb{F}_2$, so this depth model is optimistic for the attacker and conservative for the defender.)

Using Grover search, the circuit depth required to find a good permutation with a single quantum circuit application is

$$D = O\left(D_E \sqrt{p_{\text{succ}}^{-1}}\right). \quad (58)$$

However, NIST's quantum security definition imposes a maximum circuit depth constraint `maxdepth`, limiting a single quantum circuit depth to

$$D_{\text{max}} := 2^{\text{maxdepth}}. \quad (59)$$

According to Zalka’s analysis of Grover’s optimality [20], if D exceeds this constraint, the search space must be divided into N equal-sized partitions such that the Grover search on each partition satisfies the depth limit. The depth for one partition search is

$$D_N = O\left(D_E \sqrt{(p_{\text{succ}} N)^{-1}}\right). \quad (60)$$

Setting $D_N = 2^{\text{maxdepth}}$ gives

$$N = \frac{(D_E)^2 \cdot p_{\text{succ}}^{-1}}{2^{2 \cdot \text{maxdepth}}}. \quad (61)$$

Since the circuit of depth 2^{maxdepth} must be repeated N times, the total quantum gate complexity is estimated as

$$T_Q = N \cdot 2^{\text{maxdepth}} = \frac{(D_E)^2}{p_{\text{succ}} \cdot 2^{\text{maxdepth}}}. \quad (62)$$

Substituting Eqs. (56) and (57) into (62) yields the explicit form

$$T_Q = (n - k)^{2\omega_M} \cdot \frac{\binom{n}{w}}{\binom{n-k}{w}} \cdot 2^{-\text{maxdepth}}. \quad (63)$$

NIST uses the quantum gate complexity to break AES under the same `maxdepth` constraint as benchmarks, defining:

$$\begin{aligned} T_{\text{AES128}} &= 2^{170 - \text{maxdepth}}, \\ T_{\text{AES192}} &= 2^{233 - \text{maxdepth}}, \\ T_{\text{AES256}} &= 2^{298 - \text{maxdepth}}. \end{aligned}$$

Thus, a parameter set is said to reach NIST quantum security level $\ell \in \{1, 3, 5\}$ if

$$\log_2(T_Q) \geq \lambda_\ell - \text{maxdepth}, \quad (64)$$

where $\lambda_1 = 170$ (for AES-128), $\lambda_3 = 233$ (for AES-192), and $\lambda_5 = 298$ (for AES-256).

Example 10. For the *UVW128* parameter set, $(n, k, w) = (860, 430, 116)$, taking $\omega_M = 2.5$, we compute

$$\log_2(T_Q) = 172.72 - \text{maxdepth} \geq 170 - \text{maxdepth}. \quad (65)$$

From Example 10, *UVW128* achieves the 128-bit quantum security level. Analogously, for *UVW256*, $\log_2(T_Q) = 306.98 - \text{maxdepth} \geq 170 - \text{maxdepth}$, and for *UVW512*, $\log_2(T_Q) = 569.21 - \text{maxdepth} \geq 298 - \text{maxdepth}$, indicating that both satisfy the 128-bit and 256-bit quantum security levels, respectively.

4 Performance Analysis

4.1 Theoretical Performance Analysis

The computational cost of UVW is primarily concentrated in three modules:

Finite Field Arithmetic The scheme operates over $\mathbb{F}_q$. Inverses are implemented via precomputed lookup tables. Addition/subtraction is implemented in a branchless manner using mask-based conditional correction, avoiding timing variations caused by conditional branches. Vector-matrix multiplication employs 32-bit accumulators with delayed modular reduction, significantly reducing modular arithmetic overhead.

Gaussian Elimination Key generation requires Gaussian elimination on a $k \times n$ matrix, with complexity $O(k^2 \cdot n)$. The implementation exploits the zero elements in the bottom-left corner during elimination, reducing unnecessary operations. As the security level increases, k and n double, making elimination the dominant cost in key generation.

RS List Decoding The core of decryption is the Guruswami-Sudan list decoding algorithm, whose complexity is mainly determined by the interpolation and factorization steps. In practice, decoding accounts for the vast majority of the decryption time.

4.2 Performance Evaluation

Benchmarking Platform Measurements were performed on a machine running Windows 11 Pro (64-bit) with 128 GB of RAM and an Intel® Core™ i9-14900K CPU @ 3.20 GHz. For each parameter set, 1000 independent key generation, 1000 encapsulation, and 1000 decapsulation operations were executed. Each encapsulation uses a freshly generated key pair, and each decapsulation uses the corresponding ciphertext. CPU cycles for each operation were recorded. The 1000 cycles for key generation, encapsulation, and decapsulation were sorted separately, and the lower quartile, median, average, and upper quartile are reported. Meanwhile, we also report the average running time of 1000 executions of key generation, key encapsulation, and key decapsulation on the test platform.

Transmission and Storage Overhead Table 2 presents the sizes of the public key, private key, ciphertext, and session key for the recommended parameter sets at different security levels.

Table 2: Transmission and storage overhead of UVW KEM (in Bytes).

	Public key	Private key	Ciphertext	Session key
UVW128	208013	35	1032	64
UVW256	911645	35	2199	64
UVW512	4001850	67	4820	64

Reference Implementation Performance data for the reference implementation is given in Table 3. The implementation is written in C99 and compiled with gcc (MinGW-w64, 15.1.0) using flags `-O3 -std=c99 -flto -funroll-all-loops -Wall -Wextra -pedantic`.

Optimized Implementation The performance data of the optimized implementation is given in Table 4. The optimized implementation is written in C99, utilizes AVX2 instructions, and is compiled with gcc (MinGWw64, 15.1.0) with the compilation options `-O3 -std=c99 -flto -funroll-all-loops -march=native -Wall -Wextra -pedantic`.

Table 3: Performance results of the UVW KEM reference implementation.

	operation	quartile	median	average	quartile	time
UVW128	KeyGen	172957580	174424628	175360080	176185729	54ms
UVW256	KeyGen	1452749029	1462650837	1469615517	1472240134	459ms
UVW512	KeyGen	11373282624	11426363913	11437841258	11503468492	3574ms
UVW128	Encaps	414319	416606	470955	420866	0.14ms
UVW256	Encaps	1594847	1639596	1702005	1703332	0.53ms
UVW512	Encaps	11463351	11644865	11761738	11889606	3.67ms
UVW128	Decaps	414038349	426662590	428882166	439507552	134ms
UVW256	Decaps	3448351931	3523949348	3545934587	3607094531	1108ms
UVW512	Decaps	20007622666	20290008773	20483434568	20613873389	6401ms

Table 4: Performance results of the UVW KEM optimized implementation.

	operation	quartile	median	average	quartile	time
UVW128	KeyGen	82168509	82981857	83929312	84066306	26ms
UVW256	KeyGen	586859085	592341076	599684897	598800552	187ms
UVW512	KeyGen	4480146633	4506464435	4517173867	4536788037	1411ms
UVW128	Encaps	417765	421225	458819	425419	0.14ms
UVW256	Encaps	1608653	1658872	1704923	1702592	0.53ms
UVW512	Encaps	11282320	11478497	11777750	11947496	3.68ms
UVW128	Decaps	302732336	317785567	319782835	332492629	100ms
UVW256	Decaps	2616173122	2705466876	2719331496	2799935508	849ms
UVW512	Decaps	12190815341	12495559600	12607905533	12912025565	3939ms

5 Features

- **Innovation.** This work proposes the **Generalized $(U + V, U + W)$ code** [21], a novel linear code structure for constructing code-based KEM. Unlike traditional McEliece-type schemes, UVW embeds the generator matrix of a Reed–Solomon code into the private key as a trapdoor. This enables legitimate decryptors to efficiently recover the plaintext via list decoding and solving a linear system, while the adversary sees an instance indistinguishable from a random linear code under the public key. Compared with existing public-key encryption schemes based on the $(U, U + V)$ code construction [22], UVW adopts the generalized $(U + V, U + W)$ code construction, which provides more possibilities for the generated public-key matrix and thus makes it harder to distinguish the public-key matrix from a random matrix.
- **Storage Efficiency Advantages.** Using the Classic McEliece scheme from the NIST standardization process as a reference, UVW exhibits outstanding competitiveness in public key and private key sizes. For the 128-bit and 256-bit classical security levels, Classic McEliece’s public key sizes range from 255 KB to 1,326 KB, with private key sizes roughly 6.3 KB to 13.6 KB. In contrast, UVW achieves public key sizes of 203.1 KB and 890.2 KB for the same classical security levels, while the private key size is compressed to a constant **35 bytes** (for 128/256-bit levels) or **67 bytes** (for 512-bit level) via a seed-derivation strategy orders of magnitude smaller than Classic McEliece’s private key storage overhead.

References

- [1] Venkatesan Guruswami and Madhu Sudan. “Improved decoding of Reed-Solomon and algebraic-geometric codes”. In: *Proceedings 39th Annual Symposium on Foundations of Computer Science (Cat. No. 98CB36280)*. IEEE, 1998, pp. 28–37.
- [2] Mingsheng Wang. “Parameter choices on Guruswami-Sudan algorithm for polynomial reconstruction”. In: *Finite Fields Their Appl.* 13.4 (2007), pp. 877–886.
- [3] Zhe-xian Wan. “Geometry of classical groups over finite fields and its applications”. In: *Discrete Mathematics* 174.1-3 (1997), pp. 365–381.
- [4] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. “A Modular Analysis of the Fujisaki-Okamoto Transformation”. In: *Theory of Cryptography, TCC 2017*. Ed. by Yael Kalai and Leonid Reyzin. Cham: Springer, 2017, pp. 341–371. ISBN: 978-3-319-70500-2.
- [5] M. Naor and M. Yung. “Public-Key Cryptosystems Provably Secure against Chosen Ciphertext Attacks”. In: *22nd Annual ACM Symposium on Theory of Computing, STOC 1990*. ACM, 1990, pp. 427–437. ISBN: 0897913612.
- [6] Eugene Prange. “The use of information sets in decoding cyclic codes”. In: *IRE Trans. Inf. Theory* 8.5 (1962), pp. 5–9.
- [7] Pil Joong Lee and Ernest F. Brickell. “An Observation on the Security of McEliece’s Public-Key Cryptosystem”. In: *EUROCRYPT 1988*. Ed. by Christoph G. Günther. Vol. 330. LNCS. Springer, 1988, pp. 275–280.

-
- [8] Jeffrey S. Leon. “A probabilistic algorithm for computing minimum weights of large error-correcting codes”. In: *IEEE Trans. Inf. Theory* 34.5 (1988), pp. 1354–1359.
 - [9] Jacques Stern. “A method for finding codewords of small weight”. In: *Coding Theory and Applications, 3rd International Colloquium, Toulon, France, November 2-4, 1988, Proceedings*. Ed. by Gérard D. Cohen and Jacques Wolfmann. Vol. 388. LNCS. Springer, 1988, pp. 106–113.
 - [10] Daniel J. Bernstein, Tanja Lange, and Christiane Peters. “Smaller Decoding Exponents: Ball-Collision Decoding”. In: *CRYPTO 2011*. Ed. by Phillip Rogaway. Vol. 6841. LNCS. Springer, 2011, pp. 743–760.
 - [11] Anja Becker et al. “Decoding Random Binary Linear Codes in $2^{n/20}$: How $1 + 1 = 0$ Improves Information Set Decoding”. In: *EUROCRYPT 2012*. Ed. by David Pointcheval and Thomas Johansson. Vol. 7237. LNCS. Springer, 2012, pp. 520–536.
 - [12] Alexander May, Alexander Meurer, and Enrico Thomae. “Decoding Random Linear Codes in $\mathcal{O}(2^{0.054n})$ ”. In: *ASIACRYPT 2011*. Ed. by Dong Hoon Lee and Xiaoyun Wang. Vol. 7073. LNCS. Springer, 2011, pp. 107–124.
 - [13] Alexander May and Ilya Ozerov. “On Computing Nearest Neighbors with Applications to Decoding of Binary Linear Codes”. In: *EUROCRYPT 2015*. Ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9056. LNCS. Springer, 2015, pp. 203–228.
 - [14] Marco Baldi et al. “A Finite Regime Analysis of Information Set Decoding Algorithms”. In: *Algorithms* 12.10 (2019), p. 209.
 - [15] Norman Lahr et al. “Side Channel Information Set Decoding Using Iterative Chunking - Plaintext Recovery from the “Classic McEliece” Hardware Reference Implementation”. In: *ASIACRYPT 2020*. Ed. by Shiho Moriai and Huaxiong Wang. Vol. 12491. LNCS. Springer, 2020, pp. 881–910.
 - [16] Christiane Peters. “Information-set decoding for linear codes over $\mathbb{F}_q$ ”. In: *Post-Quantum Cryptography - PQC 2010*. Ed. by Nicolas Sendrier. Berlin, Heidelberg: Springer, 2010, pp. 81–94. ISBN: 978-3-642-12929-2.
 - [17] Robert Niebuhr et al. “On lower bounds for information set decoding over $\mathbb{F}_q$ and on the effect of partial knowledge”. In: *Int. J. Inf. Coding Theory* 4.1 (2017), pp. 47–78.
 - [18] Matthieu Finiasz and Nicolas Sendrier. “Security Bounds for the Design of Code-Based Cryptosystems”. In: *ASIACRYPT 2009*. Ed. by Mitsuru Matsui. Berlin, Heidelberg: Springer, 2009, pp. 88–105.
 - [19] Andre Esser and Emanuele Bellini. “Syndrome Decoding Estimator”. In: *Public Key Cryptography (1)*. Lecture Notes in Computer Science. Springer, 2022, pp. 112–141.
 - [20] Christof Zalka. “Grover’s quantum searching algorithm is optimal”. In: *Physical Review A* 60.4 (1999), p. 2746.
 - [21] Yang Yang and Fangguo Zhang. “An Efficient Encryption Scheme Based on $(U+V, U+W)$ Codes”. In: *IACR Cryptol. ePrint Arch.* 2025 (2025), p. 1149.
 - [22] Irene Marquez Corbella and Jean-Pierre Tillich. “Using Reed-Solomon codes in the $(U \mid U + V)$ construction and an application to cryptography”. In: *ISIT*. IEEE, 2016, pp. 930–934.